A DEEP LEARNING APPROACH TO SKIN CANCER DETECTION: A HYBRID PARALLEL MODEL FOR MELANOMA CLASSIFICATION

ASHKAN MOHEBALI

A THESIS FOR
THE DEGREE OF MASTER OF SCIENCE (MSc)
IN
COMPUTER ENGINEERING

CYPRUS INTERNATIONAL UNIVERSITY

INSTITUTE OF GRADUATE STUDIES AND RESEARCH

NICOSIA – 2025

A DEEP LEARNING APPROACH TO SKIN CANCER DETECTION: A HYBRID PARALLEL MODEL FOR MELANOMA CLASSIFICATION

A THESIS SUBMITTED TO
THE INSTITUTE OF GRADUATE STUDIES AND RESEARCH
OF
CYPRUS INTERNATIONAL UNIVERSITY

BY

ASHKAN MOHEBALI

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR
THE DEGREE OF MASTER OF SCIENCE (MSc)
IN
COMPUTER ENGINEERING

SUPERVISOR: ASST. PROF. DR. PARVANEH ESMAILI

NICOSIA – 2025

THESIS APPROVAL CERTIFICATE

The thesis study of Computer Engineering Program student Ashkan Mohebali with student number 22319756 titled **"**A Deep Learning Approach to Skin Cancer Detection: A Hybrid Parallel Model for Melanoma Classification**"** has been approved with unanimity of votes by the jury and has been accepted as an MSc Thesis.

Thesis Defense Date: 04/08/2025

Jury Members	**Signature**
1) Prof. Dr. Mark Stevens Chair	…….....
2) Asst. Prof. Dr. Parvaneh Esmaili	…….....
3) Prof. Dr. Member	…….....
4) Assoc. Prof. Dr. Member	…….....
5) Asst. Prof. Dr. Member	…….....

Asst. Prof. Dr. Parvaneh Esmaili …….....
Supervisor

Prof. Dr. … …….....
Head of Department / Program Coordinator
Department of …

Prof. Dr. Osman Yılmaz …….....
Director
Institute of Graduate Studies and Research

DECLARATION

Name and Surname: Ashkan Mohebali

Title of the Thesis: A Deep Learning Approach to Skin Cancer Detection: A Hybrid Parallel Model for Melanoma Classification

Supervisor: Asst. Prof. Dr. Parvaneh Esmaili

Year: 2025

I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all material and results that are not original to this work.

I hereby declare that the Cyprus International University, Institute of Graduate Studies and Research is allowed to store and make available electronically the present thesis.

This thesis will be published *via* CIU Library;

- Immediately;
- 1 year after;
- 2 years after;
- 3 years after;
- Never.

Upon submission to the Cyprus International University, Institute of Graduate Studies and Research.

Date: ……..…...

Signature: ……..…...

ACKNOWLEDGEMENTS

First, I want to thank my supervisor, Mrs. Esmaili. Her support, wise advice, and honest criticism had a significant effect on how my thesis grew and got better. I really appreciate how she was there for me during both the easy and hard parts of the process.

I also want to thank the computer-engineering department for giving me a strong background in machine learning and research, which was very important for the work I did.

I want to thank my family the most for always being there for me and encouraging me, especially when things were hard. Their faith in me has always kept me going.

Many thanks to all of my family and friends for the great talks, helpful feedback, and even the distractions that helped me stay focused while I worked on this project.

Finally yet importantly, I want to thank the people who made the ISIC datasets and the open-source community. We were able to do this work because we had access to public data and tools that make research easier for everyone to find and use.

ABSTRACT

A DEEP LEARNING APPROACH TO SKIN CANCER DETECTION: A HYBRID PARALLEL MODEL FOR MELANOMA CLASSIFICATION

Ashkan Mohebali
MSc Thesis in Computer Engineering
Supervisor: Asst. Prof. Dr. Parvaneh Esmaili

2025, [102] pages

Melanoma is a serious form of skin cancer that spreads quickly if not caught early. This project looked into how deep learning can help spot melanoma by analyzing images of skin lesions. To carry out the study, two public sources of medical image data, ISIC 2019 and ISIC 2020, were used. These datasets contain thousands of skin images, each labeled in metadata. The main goal was to develop a model that could understand the difference between the two types, detect melanoma more quickly and accurately than before.

To start, the ISIC 2019 dataset was filtered to include only images of melanoma and benign cases. This was done by reading the metadata and moving the files into separate folders based on diagnosis. After reviewing the class distribution, a clear imbalance was found, there were fewer melanoma images compared to benign ones. To help balance the dataset and improve learning, image augmentation methods were applied to increase the number of melanoma samples.

A custom model, called Berserker, was built by combining four powerful deep learning architectures: ResNet50, EfficientNetB4, DenseNet201, and Swin Transformer. These models, which were pre-trained on large-scale image data, served as frozen feature extractors. Only the final classification layers were trained, which helped shorten training time and reduce the chance of overfitting.

The model was developed using PyTorch along with data processing tools like Albumentations and Transformers. It was trained on a mixture of ISIC 2019 and ISIC 2020 training data, and its performance was measured using common evaluation metrics such as accuracy, F1-score, recall, precision, and AUC. Final testing was done using the ISIC 2019 test set.

The findings suggest that combining datasets and using a hybrid deep learning model leads to stronger performance. This work may help in developing tools that support early and more accurate melanoma diagnosis in real-world settings.

Keywords: Artificial Intelligence, Deep Learning, Hybrid Model, ISIC Dataset, Melanoma Detection, Medical Imaging, Skin Cancer

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ABBREVIATIONS

AI (Artificial Intelligence)

AUC (Area Under the Curve)

CNN (Convolutional Neural Network)

DNN (Deep Neural Network)

FC (Fully Connected)

FN (False Negative)

FP (False Positive)

ISIC (International Skin Imaging Collaboration)

LR (Learning Rate)

ReLU (Rectified Linear Unit)

ROC (Receiver Operating Characteristic)

TN (True Negative)

TP (True Positive)

CHAPTER ONE

INTRODUCTION

1.1 BACKGROUND ON SKIN CANCER AND MELANOMA

Melanoma is the most aggressive and fatal form of skin cancer, which is one of the most common kinds of cancer globally. The majority of skin cancer-related fatalities are caused by melanoma because of its high tendency for spreading, even if other skin cancers including cancer of the basal cells and squamous cell tumors often have better prognoses. Since early detection of melanoma significantly increases survival rates, immediate detection is crucial. Melanoma typically develops in melanocytes, which are the cells that produce pigment in the skin. Risk factors include UV exposure, genetics, and skin type. The growing global spread of melanoma highlights the urgent need for advanced diagnostic tools. Understanding the biology and clinical presentation of melanoma provides the foundation for developing automated detection techniques that can assist doctors in reducing misdiagnosis and improving patient outcomes.

1.2 CHALLENGES IN MELANOMA DETECTION

Despite advances in dermatology, distinguishing melanoma from benign skin lesions remains a challenging task. Even highly qualified medical professionals commonly misdiagnose melanomas as benign moles or other non-cancerous growths. The subjective visual judgment may be affected by the lesion's size, shape, color fluctuation, and illumination. Biopsies are still the gold standard, though they are invasive, time consuming and resource-intensive. Different groups have different lesion appearances, which complicates diagnosis. These challenges underscore the need for objective; automated methods that can help doctors make informed decisions. Deep learning has demonstrated promise in image-based diagnosis; however, complex models are required to effectively handle the variability and complexity of skin lesion data.

1.3 ROLE OF ARTIFICIAL INTELLIGENCE AND DEEP LEARNING IN MEDICAL IMAGING

Artificial intelligence (AI), especially deep learning, has improved medical imaging by creating systems that can successfully and independently analyze complex images. Convolutional neural networks (CNNs) and transformer models have recently demonstrated remarkable outcomes in fields such as dermatology, pathology, and radiology. By identifying important features straight from the data, these models reduce the need for human feature discovery. By increasing detection accuracy, saving time and producing more reliable results, AI systems may help physicians. However, there are still issues with medical imaging, such as lack of labeled data, imbalanced numbers of cases across classes, and challenges learning model decision-making. Specific architectures and training techniques suited to the particular difficulties of medical images are needed to overcome these challenges, especially in the detection of skin cancer.

1.4 DEEP LEARNING FOR SKIN CANCER DETECTION

One of the most important methods for identifying skin cancer from dermoscopic photos is deep learning. These methods are able to spot patterns that the human eye would miss. To differentiate between benign and malignant skin lesions, convolutional neural networks (CNNs) like ResNet, DenseNet, and EfficientNet are frequently utilized. Recently, there has also been interest in transformer-based models, such as Swin Transformer, because of its ability to capture more contextual information inside a picture.

While these individual models have achieved strong performance on public datasets like ISIC 2019 and ISIC 2020, they each have their limitations. CNNs excel at focusing on local details, whereas transformers handle long-range dependencies. Because of this, using just one type of model may result in missing key features. To address this, researchers are now combining CNNs with transformers to take advantage of both

local and global feature extraction. This combined approach is proving more reliable and effective, especially when dealing with diverse skin lesion appearances.

1.5 MOTIVATION FOR A HYBRID MODEL APPROACH

The main reason for using hybrid models is that relying on just one deep learning model has its problems. Single models usually pay attention to some features but miss others needed to classify melanoma. CNNs are good at finding local patterns, while transformers are better at understanding the bigger picture and long-distance connections. By combining multiple models in a hybrid fashion, the system can leverage each model's strengths, enabling the model to learn more and perform better on a range of data types. This technique seeks to reduce overfitting and fortifies the model against changes in lighting, image quality, and lesion appearance. These benefits suggest that hybrid models might be a better choice for clinical settings where precision is crucial. They also facilitate learning and ensure consistent performance across different datasets.

1.6 OVERVIEW OF PARALLEL HYBRID ARCHITECTURES

Several deep learning models separately process data that arrives in parallel hybrid architectures before combining the features they have acquired for classification. Unlike sequential or stacked architectures, where outputs from one model flow into another, parallel fusion maintains distinct paths for each model, reducing complexity and preventing gradient vanishing.

If the models are separated during processing, each can focus on learning distinct features without being impacted by the others. Eventually, their outputs are combined to create a more diverse and informative and diverse set of features. The model develops a more comprehensive understanding of the lesion by combining these features, which range from fine-grained local textures to broader image context.

This approach is especially helpful in melanoma detection, where lesion shapes, sizes, and colors vary widely. Additionally, the modular design of parallel fusion makes it easier to experiment with different model combinations and training strategies, improving both flexibility and efficiency in development.

1.7 THESIS OBJECTIVE

One of the most deadly types of skin cancer is melanoma, and increasing patient survival rates requires early identification. Even experienced doctors find it difficult to differentiate benign tumors from cancer. Through the use of transformer-based models and several pretrained CNNs, this thesis aims to develop a powerful deep learning framework that improves melanoma detection accuracy.

In particular, a fully connected neural network for classification is proposed after a parallel hybrid architecture that combines features from ResNet50, EfficientNetB4, DenseNet201, and Swin Transformer. "A Deep Learning Approach to Skin Cancer Detection: A Hybrid Parallel Model for Melanoma Classification" the title chosen, reflects the work's twin purpose of addressing a critical healthcare issue and providing a unique multi-backbone fusion technique to enhance diagnostic performance.

1.8 SIGNIFICANCE AND IMPACT OF THE STUDY

This thesis studies an important problem in healthcare by using a modern deep learning method to help diagnose melanoma more accurately. Melanoma is increasing in many countries, and finding it early can greatly improve the chance of successful treatment. The system developed in this research can support dermatologists by giving faster and more correct results. This can help reduce unnecessary biopsies and lower healthcare costs.

This study's model combines many concurrent deep learning networks. This makes it possible for the algorithm to glean more useful information from images of skin

lesions. For different patient types, the model may therefore produce more precise and reliable findings.

The study also gives new ideas for future work in medical image analysis. At the same time, it makes current AI tools for diagnosis better. This method can support doctors in their daily practice and increase trust in using AI to find skin cancer.

1.9 OUTLINE OF THE THESIS STRUCTURE

This thesis has five main chapters. The first chapter, introduction, explains the background, the goals of the research, and why the topic is important. The second chapter, which is literature review, looks at other research on melanoma detection and different deep learning models. In the third chapter, the dataset, image processing steps, model design, and training details are described.

The fourth chapter shows the results, compares the model with other methods, and talks about the findings. Finally, the last chapter gives a short summary, points out the limits of the study, and suggests ideas for future work. This structure helps the reader understand the whole process, from the idea to the results and what they mean.

1.10 SUMMARY OF CONTRIBUTIONS

This thesis aims to improve the automated diagnosis of skin cancer by presenting a novel model that integrates four well-known architectures: ResNet50, EfficientNetB4, DenseNet201, and Swin Transformer. Each of these models contributes distinct feature extraction capabilities, which are then combined and processed by a fully connected neural network for more accurate melanoma classification. Using the ISIC 2019 and ISIC 2020 benchmark datasets, the proposed approach was tested and showed results equal to or better than other methods. To increase the model's reliability for clinical use, the confidence scores of its predictions were also examined.

In summary, this work offers a strong and flexible framework that can improve melanoma detection and be extended to other medical imaging problems.

CHAPTER TWO

LITERATURE REVIEW

2.1 OVERVIEW OF MELANOMA AND COMPUTER-AIDED DIAGNOSIS

Melanoma is a dangerous type of skin cancer that develops from melanocytes, the cells that produce skin pigment. Detecting melanoma early is very important for improving survival chances, but it remains difficult because it can look very similar to benign skin spots. To help doctors analyze dermoscopic images more effectively, computer-aided diagnostic (CAD) systems have been developed over the years. These systems initially used traditional machine learning methods but have gradually shifted toward more advanced deep learning techniques. Recent progress in computer vision and deep learning has made these CAD tools more dependable. For example, Ye et al. (2024) found that deep learning models for melanoma detection had a pooled AUC of 0.92, with sensitivity and specificity of 0.82 and 0.87, respectively, according to a systematic review and meta-analysis. The study also showed that the best diagnostic performance, which approaches dermatologist-level accuracy, is provided by hybrid architectures that combine segmentation and classification.

2.2 DEEP LEARNING IN SKIN CANCER DETECTION

One of the most effective methods for detecting skin cancer is deep learning. Since they can recognize intricate patterns in images, convolutional neural networks (CNNs) in particular have demonstrated exceptional performance. The ability of deep learning models to classify melanoma from dermoscopic images has been shown in numerous studies around the world.

For instance, Sancar (2024) applied fine-tuned MobileNet and DenseNet121 models with ensemble learning to improve classification results. His model reached an accuracy of 93.03% on the ISIC 2018 dataset, showing that lightweight and deeper models combined can deliver robust performance. Melanoma Classification Through

Deep Ensemble Learning and Explainable AI, an ensemble model put forth by Perera et al. (2024), employed five CNNs, including ResNet and DenseNet, to classify melanoma with an accuracy of 93.1% and an F1-score of 0.92.

Another significant work is by Albahli (2025), who developed a real-time melanoma detection system using YOLOv8. This model achieved a high mAP@0.5 of 98.6% and a Dice score of 0.92 by integrating segmentation modules and advanced augmentation techniques like CutMix and Mosaic. These results demonstrate how object detection models can also be adapted for skin cancer classification.

2.3 HYBRID AND ENSEMBLE METHODS IN DEEP LEARNING

To make the most of different neural network types, hybrid models often combine segmentation and classification modules. By capturing both low-level and high-level features from dermoscopic images, these networks can improve detection accuracy. For example, Zhang and Chaudhary (2024) introduced a hybrid framework combining U-Net for segmentation with EfficientNet for classification, achieving 99.01% accuracy on ISIC-2020 data, showing that integrating segmentation can vastly improve results.

Using attention-based fusion across ConvNeXt, EfficientNetV2, and Swin Transformer, another hybrid model, SkinEHDLF from Lilhore et al. (2025), achieves 98.6% accuracy and 99.7% AUROC, a performance boost of about 7.9%.

Meanwhile, Perera et al. (2024) developed an ensemble using ResNet-101, DenseNet-121, and Inception v3, enhanced with explainable-ai techniques (SHAP), but the model does not include MobileNet + DenseNet121 and focuses more on interpretability rather than the specific structures you mentioned.

2.4 COMPARISON OF FEATURE EXTRACTION APPROACHES

In computer-aided diagnostic (CAD) systems, extracting useful features from images is one of the most important steps for detecting skin cancer accurately. Traditional

methods often use handcrafted features, such as texture patterns (like GLCM and LBP), color data, and shape information, which are designed by experts. Conversely, deep learning models, particularly convolutional neural networks, or CNNs, can automatically extract intricate patterns from dermoscopic images without the need for human design.

Some recent studies show that combining both types of features can give better results than using only one. Ali and Ragb (2021), for instance, developed a model that combines CNN outputs with manually created features such as GLCM and LBP. They added the manually created data straight into the CNN's fully connected layers, skipping the usual processes of image preprocessing and segmentation. This method led to a 6.8% improvement in accuracy on the ISIC-2018 dataset, showing that this fusion strategy can be helpful.

In another study, Mateen et al. (2024) developed a deep learning system that includes U-Net for segmentation, Inception-ResNet-v2 for extracting visual features, and Vision Transformers to improve classification. They tested it on the ISIC-2020 and HAM10000 datasets and reached 98.65% accuracy, with 99.2% sensitivity and 98.03% specificity.

Similarly, Grignaffini et al. (2023) introduced an approach where handcrafted features were added directly into the CNN model. This method avoided standard preprocessing steps and used both traditional and deep learning features together. Their findings demonstrated that the model continued to function well while streamlining the workflow.

All things considered, these studies imply that combining handcrafted and deep learning features using various fusion techniques can result in melanoma detection systems that are more accurate and dependable.

2.5 SEGMENTATION-CLASSIFICATION FRAMEWORKS

In order to separate the lesion area from the skin around it, segmentation is a crucial pre-processing step in skin lesion analysis. This improves classification performance by lowering background noise. Segmentation improves diagnostic prediction reliability and interpretability by directing the model's attention to the lesion itself. In recent years, researchers have increasingly adopted integrated frameworks that combine both segmentation and classification into unified pipelines, often yielding state-of-the-art results.

One prominent example is ScaleFusionNet, introduced by Qamar et al. (2025), which combines multi-scale feature extraction with transformer-guided attention to segment skin lesions. The model utilizes Swin Transformer blocks and a Cross-Attention Transformer Module (CATM) to fuse local and global features, capturing fine lesion boundaries with high accuracy. Their experiments showed impressive Dice scores of 92.94% on ISIC-2016 and 91.65% on ISIC-2018, confirming the model's ability to generalize across different datasets.

In another study, Hasan and Rifat (2025) developed a segmentation-assisted ensemble system for classification using the new ISIC 2024 SLICE-3D dataset. Their approach employs two distinct deep learning models, EVA02 Vision Transformer and a convolutional variant called EdgeNeXtSAC, for image-level feature extraction. These predictions are then fused with patient metadata using a Gradient Boosted Decision Tree (GBDT). To address class imbalance, they generated synthetic malignant lesions using diffusion models. The system achieved a partial AUC of 0.1755 above 80% TPR, demonstrating strong performance under real-world diagnostic conditions.

Similarly, Akter et al. (2025) presented a hybrid deep learning model that integrates InceptionV3 and DenseNet121 networks. Instead of a typical early or late fusion approach, their model combines features using a weighted sum strategy. The model reached 92.27% accuracy, with 92.33% sensitivity, 92.22% specificity, and an F1-

score of 91.57% on a balanced dataset, showing that such hybrid classification models can perform reliably when trained on quality segmentations.

While many frameworks rely on dermoscopic data, Nandi (2025) reported on SkinEHDLF, a cutting-edge hybrid model trained on large-scale non-dermoscopic 3D total-body images from ISIC 2024. The model uses ConvNeXt, EfficientNetV2, and Swin Transformer backbones, fusing their outputs via an attention-based fusion layer. SkinEHDLF achieved 99.8% AUROC for binary classification and 98.6% accuracy on multi-class tasks. These numbers represent a significant leap in real world skin cancer classification, especially when paired with segmentation modules that further localize lesion boundaries.

These studies consistently show that combining segmentation and classification, especially with advanced architectures like transformers, ViTs, and metadata fusion can dramatically improve the performance of skin cancer detection systems. Segmentation narrows the visual focus, while classification models capitalize on refined features, making these integrated frameworks both efficient and clinically practical.

2.6 ADDRESSING DATA IMBALANCE AND BIAS

Class imbalance is a fundamental challenge in melanoma detection, as malignant cases are far less frequent than benign lesions. This imbalance can bias deep learning models toward over-classifying the majority benign class, reducing sensitivity for melanoma, a critical problem in clinical applications. Consequently, researchers have developed various strategies to mitigate this issue and improve diagnostic fairness and robustness.

One promising approach is feature fusion combined with fairness-aware modeling. Abdulredah et al. (2025) proposed SkinWiseNet, a model designed to reduce bias across demographic subgroups by integrating features from multiple datasets and modalities. This multi-source feature fusion enhances the model’s ability to generalize across diverse skin tones and hair types, improving fairness and accuracy

simultaneously. Although not explicitly focused on normalization techniques, SkinWiseNet's fusion strategy effectively mitigates biases associated with underrepresented groups, making it a valuable direction for addressing demographic imbalance in melanoma datasets.

Another effective method, as shown by el Mrabet et al. (2025), involves classical class rebalancing techniques such as data augmentation, oversampling, and class weighting. Their hybrid deep learning system trained on the ISIC 2024 SLICE-3D dataset applied these strategies to counteract the inherent imbalance in melanoma detection. Notably, they generated synthetic malignant lesion examples via advanced augmentation methods to enrich the training data, which improved model robustness and sensitivity. By combining convolutional networks with metadata features, their approach achieved strong performance in real-world clinical settings, demonstrating the practical benefits of integrated balancing strategies.

By using class weighting and targeted data augmentation during training to address class imbalance, the methodology conforms to accepted practices. Minority melanoma samples are given priority for augmentation, and loss functions are modified to highlight infrequent but crucial positive cases. These steps are used to improve clinical applicability by lowering bias and increasing the reliability of melanoma detection through meticulous model design and validation.

2.7 MODEL INTERPRETABILITY AND CONFIDENCE SCORING

In addition to making accurate predictions, AI systems must also provide information about how reliable those predictions are in order to be used in clinical environments. This is particularly important when it comes to melanoma detection, as misclassifications can have fatal outcomes. As a result, confidence scoring is now a crucial part of trustworthy diagnostic AI.

Perera et al. (2024) developed a hybrid ensemble model combining ResNet101, DenseNet121, and InceptionV3 to enhance diagnostic performance for melanoma

classification. Beyond accuracy, their work emphasized the value of model confidence. They offered a way to evaluate classification certainty by examining the softmax output probabilities of the ensemble's predictions. Their results highlight the fact that models with calibrated confidence scores are better suited for practical application, particularly when incorporated into decision-support systems that allow for the flagging of uncertain outputs for additional clinical review.

Similarly, to improve model trustworthiness, Singh et al. (2022) presented SkiNet, a strong two-stage deep learning framework that includes uncertainty estimation. Their pipeline generated multiple predictions per input image using Test-Time Augmentation (TTA) and Monte Carlo Dropout during inference. The model then averaged the softmax probabilities across augmented variants to produce a final confidence score. This method enabled the identification of high-variance, low-confidence outputs that should be escalated to human experts. SkiNet's results showed that this uncertainty-aware approach improves safety and reliability, especially when working with ambiguous or borderline cases in melanoma detection.

All of these studies show that a crucial component of developing AI models for clinical deployment is the inclusion of probabilistic confidence measures like ensemble averaging, dropout-based uncertainty sampling, and softmax thresholding. Confidence scoring makes it easier to tell the difference between accurate and uncertain predictions and makes it safer to implement automated systems into dermatological workflows.

2.8 EVALUATION METRICS IN RECENT STUDIES

To judge how well deep learning models detect melanoma, researchers rely on different evaluation metrics. While accuracy is a basic one, it is often not enough, especially when data is imbalanced. Therefore, most recent papers also report sensitivity, specificity, F1-score, AUROC, and other detailed measures to give a clearer view of model performance.

For example, in their study, Lilhore et al. (2025) introduced SkinEHDLF, a hybrid model tested on both binary and multi-class datasets. It got an AUROC of 99.8% and an accuracy of 98.76% in binary classification. For multi-class tasks, the accuracy was 98.6%, with precision and recall scores close to 98%. To completely comprehend the behavior of the model, the authors also examined MCC, FPR, and MSE.

Albahli (2025) focused on real-time detection using a YOLOv8-based system. His model was tested with object detection metrics like mAP@0.5, which reached 96.7%, and segmentation accuracy using Dice scores, which hit 0.92. These metrics show how well the model can not only detect but also segment melanoma lesions in images.

Another study by Zhang (2024) presented a hybrid deep learning framework and reported a classification accuracy of 99.01% on the ISIC 2020 dataset. Although there were few other metrics mentioned in the paper, the high accuracy indicates the model performed well.

Finally yet importantly, Perera et al. (2024) examined a number of studies and noted that high-quality models typically report multiple metrics. They emphasized the usefulness of accuracy, precision, recall, specificity, F1-score, and AUROC in assessing a model's performance in actual diagnosis.

Together, these studies show that relying on multiple metrics is now standard and necessary. They help researchers and doctors know not just if a model is right, but how and when it makes mistakes, especially in cases that matter most, like detecting melanoma early.

2.9 ADVANTAGES OF PARALLEL HYBRID MODELS

As deep learning continues to grow, especially in complex areas like medical image analysis, it has become clear that using only one type of model, even if it is very advanced, might not be enough to get the best results. Because of this, more researchers are now turning to parallel hybrid models. These models bring together different types

of deep learning networks so they can work side by side. By combining their strengths, they help the system give better, more accurate and more dependable results than using a single model alone.

One big reason why parallel hybrid models are useful is that they make use of different ways models extract features from data. Each deep learning model is good at spotting certain patterns. For example, Convolutional Neural Networks (CNNs) are excellent at finding local details like shapes, edges, or textures. In contrast, Transformer models use attention mechanisms to understand broader patterns and relationships across the whole image (Lilhore et al., 2025). When both kinds of models work together in parallel, and their outputs are combined, the system gets a fuller picture of what is in the image. A good example is the SkinEHDLF model, which uses ConvNeXt, EfficientNetV2, and Swin Transformer. Because of this setup, it benefits from strong feature detection, good scaling, and attention over the whole image, making it highly reliable for skin cancer classification (Lilhore et al., 2025).

Another advantage of parallel hybrid models is their strength and flexibility. A single model might not always handle data differences, image noise, or unusual cases well. But when multiple models work together, they can help cover each other's weaknesses. When dealing with patients from diverse backgrounds or when the dataset contains biases, this combination of feature paths aids the system in learning from multiple perspectives (Abdulredah et al., 2025). This is very important in real clinical situations, where images and skin features vary a lot between patients. These models are better at ignoring unimportant parts of the image and focusing on what matters, which leads to fewer mistakes and better trust in the results.

In addition, using parallel hybrid models can lead to better overall performance. For example, a system that uses U-Net to focus on segmentation and EfficientNet for classification can do very well in detecting melanoma (Zhang, 2025). Another method is ensemble learning, where multiple models like MobileNet and DenseNet121 are trained and used together. This helps each model focus on different features and

reduces the risk of one model making too many mistakes on its own (Sancar, 2024). Some advanced designs, like ScaleFusionNet, go even further by using Cross-Attention Transformers and Adaptive Fusion Blocks. These help with both feature extraction and merging outputs, improving segmentation and showing how combining smart designs from different models can give better results (Qamar et al., 2025).

In conclusion, parallel hybrid models are an effective way to overcome the difficulties in deep learning, particularly when it comes to tasks like medical diagnosis. Compared to using a single model, these systems can provide more accurate, reliable, and trustworthy results by combining various models and carefully integrating the lessons learned by each.

2.10 SUMMARY AND RESEARCH GAP

In recent years, deep learning has become one of the most powerful tools for detecting melanoma from skin images. Many studies have explored different architectures such as CNNs, Transformers, and Vision Transformers, and found that these models can often reach high levels of accuracy. Some models, like those developed by Lilhore et al. (2025) and Albahli (2025) have achieved impressive scores across various metrics, AUROC, mAP, accuracy, and Dice scores. These results prove that deep learning can be very effective in both classification and segmentation tasks for skin cancer detection.

Interestingly, parallel hybrid models are becoming more popular. These models combine different backbones, like CNNs and Transformers, to take advantage of both local and global features in images. For example, the SkinEHDLF model used ConvNeXt, EfficientNetV2, and Swin Transformer to improve both accuracy and generalization. Other studies, such as those by Sancar (2024) and Qamar et al. (2025), used ensembles or fusion blocks to boost performance even more. This shows that combining diverse model types in a parallel setup can make a big difference.

Evaluation metrics also vary across studies. Most commonly, researchers use accuracy, precision, recall, F1-score, and AUROC. Some works, like the one by Zhang (2025), report only overall accuracy, while others, such as Lilhore et al. (2025), give a full set of metrics, including TPR, FNR, and even MSE. This shows a growing awareness of the need to evaluate models from multiple angles, especially when dealing with sensitive medical tasks.

Another big issue is data imbalance. Melanoma images are far fewer than benign ones, which makes it harder for models to learn to recognize the dangerous class. Studies like Abdulredah et al. (2025) and el Mrabet et al. (2025) proposed using data augmentation, class weighting, and fairness-aware fusion methods to reduce this problem. These methods are key in making sure models do not perform poorly on minority classes or underrepresented patient groups.

Still, there are a few gaps. First, many recent models focus either many recent models focus either on segmentation or classification, but not both. In addition, not all models report confidence scores, which are important for deciding when a prediction should be trusted. While some papers, like Perera et al. (2024) and Singh et al. (2024), use confidence-based outputs to improve reliability, this is not a universal practice yet.

In short, while there has been a lot of progress, more work is needed to bring all the pieces together. Most research still lacks unified models that are strong, explainable, and fair at the same time. The current thesis tries to help fill this gap. By using a parallel hybrid model with multiple frozen backbones and a trainable classification head, it aims to boost accuracy and reduce bias, especially for melanoma cases. Confidence-based prediction thresholds and careful validation are also applied to increase clinical trust. Hopefully, this work can be a step toward more robust and practical AI tools in dermatology. In addition, **Table 2.1** shows the full comparison of some of the references used in this thesis below.

Table 2.1: Comparative summary of some of the deep learning models

Reference	Model / Technique	Dataset(s)	Key Results	Main Innovation
Abdulredah et al. (2025)	SWNet: multi-path deep feature fusion	HAM10000, ISIC-2019, ISIC-2020	Acc 99.86%, F1 99.95%	Reduces bias via feature fusion; Grad-CAM explainability
Akter et al. (2025)	InceptionV3 + DenseNet121 (weighted ensemble)	ISIC + external dataset	Acc 92.3% → 93.5%, F1 91.6% → 92.3%	CNN ensemble with external validation for greater generalization
Albahli (2025)	YOLOv8-based unified detection + segmentation	ISIC-2020, HAM10000, PH2	mAP@0.5 98.6%, Dice 0.92, IoU 0.88	Real-time melanoma detection and segmentation in a single stage
Ali & Ragb (2021)	CNN + handcrafted (GLCM, LBP)	Dermoscopic dataset	Balanced Acc 92.3%	Simple hybrid with improved accuracy
Lilhore et al. (2025)	SkinEHDLF: ConvNeXt + EfficientNetV2 + Swin with attention fusion	ISIC-2024 TBP (3D images)	Acc 98.76%, Precision 97.9%, Recall 97.3%, AUROC 99.8%	Adaptive fusion across CNN and transformer features to reduce false positives
Mateen et al. (2024)	U-Net + InceptionResNet-v2 + Vision Transformer	ISIC-2020, HAM10000	Acc 98.65%, Sens 99.2%, Spec 98.0%	Hybrid segmentation-classification pipeline with transformer-enhanced refinement

Table 2.1 (Continued): Comparative summary of some of the deep learning models

Perera et al. (2024)	Ensemble: ResNet101, DenseNet121, InceptionV3 + XAI	ISIC dermoscopic images	F1 ≈ 95%	Combines CNN ensemble with explainable AI for transparent melanoma predictions
Grignaffini et al. (2023)	CNN + handcrafted texture features (no segmentation/preprocessing)	ISIC-2016, ISIC-2018, ISIC-2019	Acc ≈ 95% (on ISIC-2016)	Fuses manual texture features into CNN, bypasses segmentation for speed
Hasan & Rifat (2025)	Hybrid: ViT (EVA02) + CNN + segmentation-assisted classifier + GBDT + synthetic lesions	ISIC-2024 SLICE-3D TBP	pAUC@80% TPR = 0.1755	Integrates metadata and synthetic lesion augmentation with hybrid architecture
Qamar et al. (2025)	ScaleFusionNet: Swin transformer multi-scale FusionBlock for segmentation	ISIC-2016, ISIC-2018	Dice 92.9%, 91.6%	Multi-scale feature fusion and transformer guidance for lesion segmentation
Sancar (2024)	Fine-tuned MobileNet + DenseNet121 ensemble	ISIC lesions dataset	Acc 93.0%, Prec & Recall & F1 ≈ 92.4%	Efficient CNN ensemble for early melanoma detection

Table 2.1 (Continued): Comparative summary of some of the deep learning models

el Mrabet et al. (2025)	MobileNetV2 / EfficientNet-B0 / ResNet-18 / DenseNet-121 + metadata fusion	ISIC-2024 SLICE-3D TBP	MobileNetV2: Acc 97.7%, Rec 99.2%; EfficientNet-B0: Acc 97.2%, Rec 98.5%	Lightweight telemedicine-ready CNN with lesion metadata inclusion
Nandi, S. (2025)	SkinEHDLF – hybrid model combining ConvNeXt, EfficientNetV2, Swin Transformer with attention-based feature fusion	ISIC 2024 total-body 3D TBP dataset (401,059 images)	AUROC 99.8%, Accuracy 98.6%, Precision 97.9%, Recall 97.3%	Triple-backbone adaptive fusion architecture; scalable, clinically-focused deployment

CHAPTER THREE

METHODOLOGY

3.1 OVERVIEW OF THE PROPOSED PIPELINE

The goal of this research is to design and evaluate a deep learning model for skin cancer detection, specifically melanoma classification. To achieve this, a complete machine-learning pipeline was developed, beginning from dataset preparation and ending with model evaluation. The main innovation in this work is the use of a hybrid deep learning model named Berserker, which combines four different neural network backbones to improve classification performance.

The overall pipeline includes several essential stages. First, two publicly available datasets ISIC 2019 and ISIC 2020 were used as sources of dermoscopic images. Because of differences in image format, class labeling and structure, each dataset needed a different set of preprocessing steps. The problem of class imbalance between melanoma and benign samples was addressed by merging and balancing the two datasets after they had been cleaned and formatted.

The next part of the pipeline involved building the Berserker model architecture. This model combines four well-known pre-trained networks, which are ResNet50, EfficientNetB4, DenseNet201, and Swin Transformer into a single hybrid model. All backbones were frozen, meaning their internal weights were not updated during training. Only the custom classification head was trained on the combined dataset.

After building the model, it was trained using a standard supervised learning approach. Cross-entropy loss was used as the loss function, and also the AdamW optimizer was applied for training. The performance of the model was measured using a separate test set from ISIC 2019. Metrics such as accuracy, sensitivity, precision, and F1-score were used to evaluate results. The pipeline also included threshold tuning and test-time augmentation to further improve model reliability.

3.2 DATASET DESCRIPTION AND SOURCES

This research uses two well-known dermoscopic image datasets, which are ISIC 2019 and ISIC 2020. Both datasets belong to the International Skin Imaging Collaboration (ISIC) archive, which is frequently used in research on melanoma detection and skin lesion analysis. These datasets, which include thousands of photos of skin lesions, have been used in numerous machine learning contests and studies.

The ISIC 2019 dataset includes over 25,000 RGB images of skin lesions and is originally multiclass. It contains eight diagnostic categories such as melanoma, nevus, basal cell carcinoma, and others. However, for this study, we focused only on the melanoma and benign (nevus) categories. The other classes were removed to convert the problem into binary classification. Melanoma was labeled as the positive class, and all benign samples were treated as the negative class. This filtering made the dataset suitable for training a binary classification model.

The ISIC 2020 dataset consists of over 33,000 dermoscopic images stored in DICOM format. These images were provided without test labels, meaning they could not be used for final evaluation. In our pipeline, ISIC 2020 was used only for training and validation. The dataset is heavily imbalanced, with fewer than 1000 melanoma cases and the rest labeled as benign. This imbalance did indeed require special handling during preprocessing.

Both datasets were downloaded and stored in Google Drive for easy access through Google Colab. All together, these datasets offered a large and diverse sample of skin lesion images for training a deep learning model. Combining them helped improve model generalization across image sources and conditions. The following sections explain how each dataset was cleaned, resized, normalized, and prepared for model training. Since one of the cases the model was used for was training the model on both the ISIC 2019 and ISIC 2020 training sets, the general look of the preprocessing of ISIC 2019 and ISIC 2020 is shown in the **Figure 3.1**, which simply shows the case

that both datasets will be combined together for being used for training the model below.

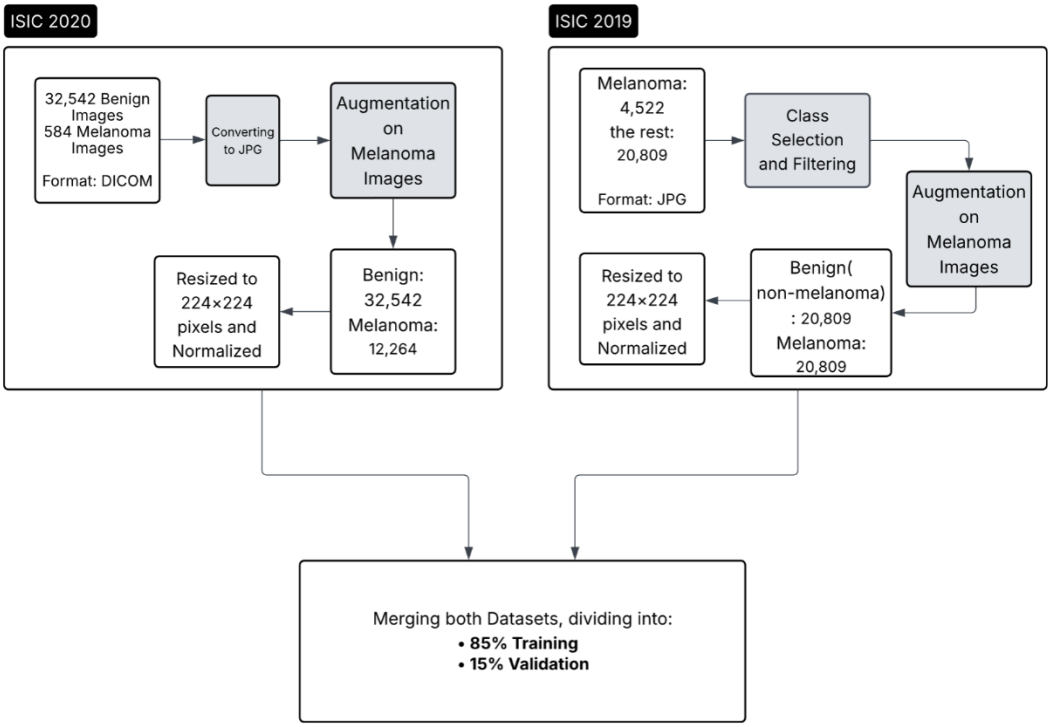

Figure 3.1: General looking to the preprocessing of ISIC 2019/2020 training sets

3.3 DATA PREPROCESSING PIPELINE

One crucial step in the pipeline is the preprocessing phase. Separate procedures were required to get the ISIC 2019 and ISIC 2020 datasets ready for model training, and the reason is, they differ in format, structure, and class labels. The preprocessing done on both datasets prior to integration is described in this section.

For ISIC 2019, the original dataset contains eight skin lesion categories. We filtered it to include only melanoma and benign (nevus) classes. All other classes were removed. The selected images were resized to 224×224 pixels in RGB format and normalized by dividing pixel values by 255, bringing them into a [0, 1] range. The normalized

images were saved in JPG format for efficient loading, organized into two subfolders: Melanoma and Benign. As for the ISIC 2019 testing set, the **Figure 3.2** briefly shows the general structure and its preprocessing pipeline below.

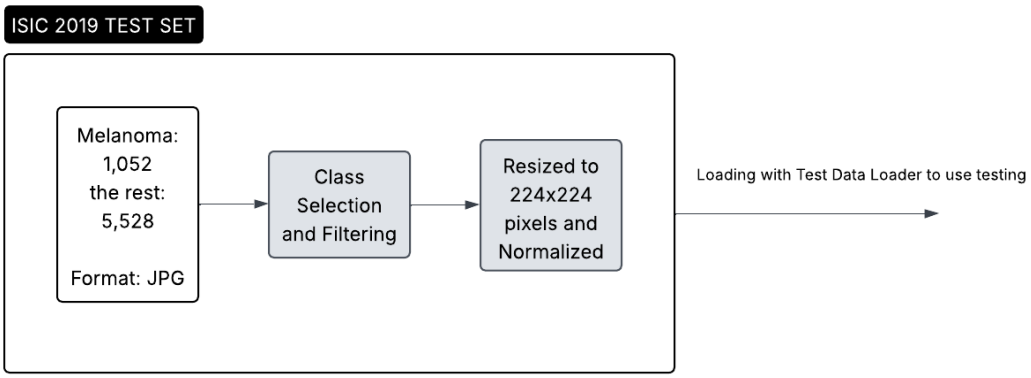

Figure 3.2: General looking to the preprocessing of the ISIC 2019 testing set

For ISIC 2020, the original images were stored in DICOM format. Therefore the image data and discarded the metadata were extracted, which was not needed for visual classification. Also, These images were resized to 224×224×3 pixels and normalized in the same way. The processed photos were arranged into subfolders labeled Melanoma and Benign and saved in JPG format.

Python scripts in a modular format were used to preprocess both datasets in Google Colab. Reproducibility and simple preprocessing pipeline updates were made possible by this configuration. Two clean, normalized, and structured datasets were produced as a result, which could then be combined and used to train the Berserker model.

3.4 DATA AUGMENTATION AND CLASS BALANCING

One of the biggest challenges in medical imaging is class imbalance. In both ISIC 2019 and ISIC 2020, melanoma images are much fewer than benign cases. This imbalance can cause a model to perform poorly on melanoma detection, which is the most

important class in this study. To address this, data augmentation was applied during the preprocessing phase, but only to melanoma images.

The augmentation process was performed using the Albumentations library, which provides many image transformation options. In our case, we applied horizontal and vertical flips, rotations, and slight changes in brightness and contrast. These transformations simulate real-world variations in dermoscopic images and help the model learn features that are more robust.

The target was to increase the number of melanoma images to match the number of benign images, 20,809 in total. The augmented images were saved with new filenames and stored in the same directory as the original melanoma cases. This method did make it possible to create a balanced dataset without affecting the benign class.

It is important to note that augmentation was applied only during preprocessing, not during model training. This method prevented the introduction of randomness during training epochs and guaranteed that the training data stayed consistent across runs. By addressing the class imbalance before training, the chances of the model learning meaningful patterns equally from both classes were improved.

3.5 DATA PARTITIONING STRATEGY

After preprocessing and balancing, the next step was to split the dataset into three sets, which are training, validation and test sets. Hence, the partitioning is important to ensure the model learns from one part of the data, is validated on another, and is tested on completely unseen images.

TA single dataset with balanced melanoma and benign classes was created by combining the preprocessed ISIC 2019 and ISIC 2020 training sets. As it was mentioned in **Figure 3.1**, the combined dataset was divided into 15% for validation and 85% for training at random. This allowed for hyperparameter adjustments without affecting the test set and model performance monitoring during training.

The ISIC 2019 test set was used as the final evaluation dataset. This set includes ground truth labels, making it suitable for measuring real performance. Since the ISIC 2020 test set does not include labels, it was not used for testing. The consistent use of the ISIC 2019 test set across all experiments allowed fair comparison of different models and training setups.

Splitting was done using PyTorch's random_split function to ensure randomness while maintaining reproducibility. Additionally, it was made sure that the distribution of classes in the training and validation sets was balanced. By using this partitioning technique, the model evaluation was guaranteed to be accurate and unaffected by overfitting to the validation set or data leakage.

3.6 CUSTOM DATA LOADERS AND TRANSFORMATION PIPELINE

At the moment, that the datasets were preprocessed and partitioned, they needed to be loaded efficiently during training and validation. To achieve this, a custom PyTorch Dataset class was created. This class, called ISICCombinedDataset, had the duty to read the images from disk, assign labels and apply necessary transformations before passing the data to the model.

As the **Figure 3.1** shows, the dataset class could support both ISIC 2019 and ISIC 2020 formats. It accessed images from the Melanoma and Benign folders and assigned binary labels, which means 1 for melanoma and 0 for benign. During data loading, the class applied transformations using the Albumentations library. These included resizing the image to 224×224, normalizing the pixel values to [0, 1], and converting the image to a PyTorch tensor.

In this work, PyTorch's DataLoader was successfuly working to batch the data and load it efficiently. Data was shuffled at each epoch to prevent the model from memorizing the order of the samples. Multi-threaded loading was enabled by setting the number of workers to 12 and using a batch size of 128 for training. Separate dataset classes were also used to load the test and validation data, but only minor adjustments,

such as resizing and normalization, were made. In the **Figure 3.3**, also there are some visualized sample training images after being loaded with the dataloaders.

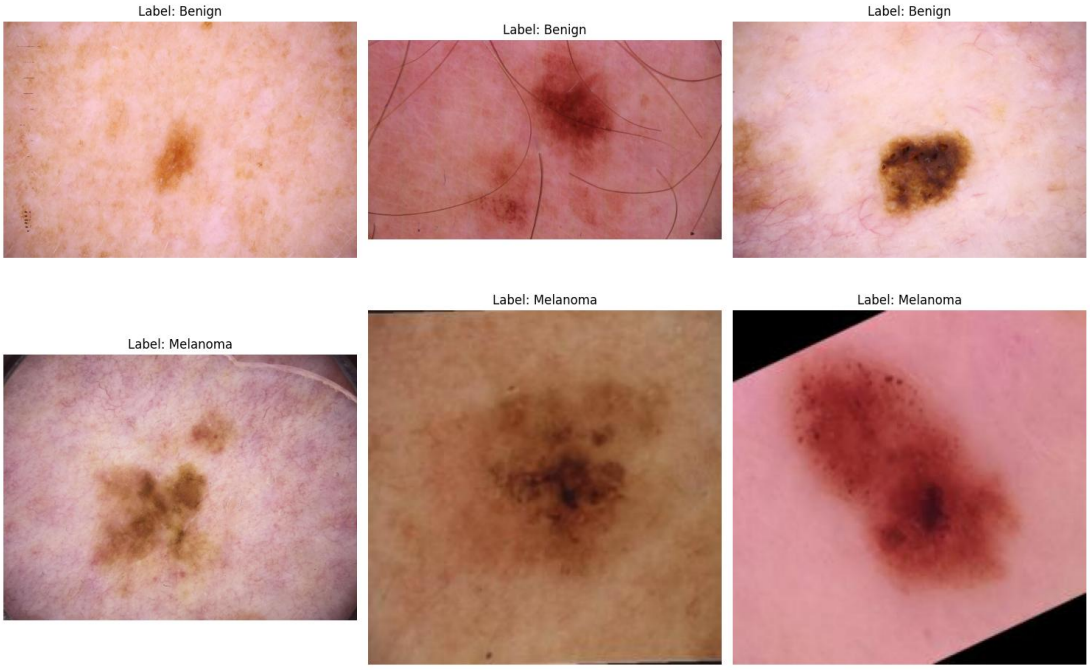

Figure 3.3: Sample images with the shape: torch.Size ([128, 3, 224, 224]) and labels shape: torch.Size ([128])

Training was quick, stable, and memory-efficient thanks to this modular and reliable loading pipeline, particularly when utilizing Google Colab with constrained GPU and RAM. Additionally, the use of custom loaders made it simple to integrate with other experiments, like applying test-time augmentation or testing individual backbones. Overall, this setup created a flexible and reproducible foundation for managing input data.

3.7 BERSERKER: HYBRID MODEL ARCHITECTURE

The name of the central model used in this research is Berserker. It is a hybrid deep learning architecture designed to improve binary classification of skin lesions. Berserker is built by combining four powerful convolutional and transformer-based

networks into a single model. These are ResNet50, EfficientNetB4, DenseNet201, and Swin Transformer.

Each of these backbones has different strengths. For instance, ResNet50 uses skip connections to improve gradient flow. On the other hand, EfficientNetB4 is optimized for high accuracy with low computational cost. While DenseNet201 connects layers directly to promote feature reuse, Swin Transformer captures long-range dependencies using windowed attention and making it effective in detecting texture and pattern variations. All of these four networks were pre-trained on ImageNet.

In the Berserker model, each backbone receives the same input image and extracts high-level features in parallel. Their final feature vectors are flattened and concatenated into one long vector. This combined feature representation is then passed through a custom classification head made of fully connected layers. Batch normalization and dropout are used between layers to improve generalization and prevent overfitting.

An important aspect of this design is of course that all backbone weights are frozen during training. Only the classification head is trained, allowing the model to focus on learning how to combine the extracted features without overfitting. This method keeps the advantages of robust pre-trained features while cutting down on training time and memory usage.

PyTorch and the timm and transformers libraries are used in the construction of Berserker. Its modular architecture makes it easy to experiment with different fusion processes and change backbones. This architecture provides a real fair and balanced trade off between model complexity and performance and it was organically developed specifically for the challenges of medical picture categorization. The **Figure 3.4** shows the general looking of the model and what it does to the images.

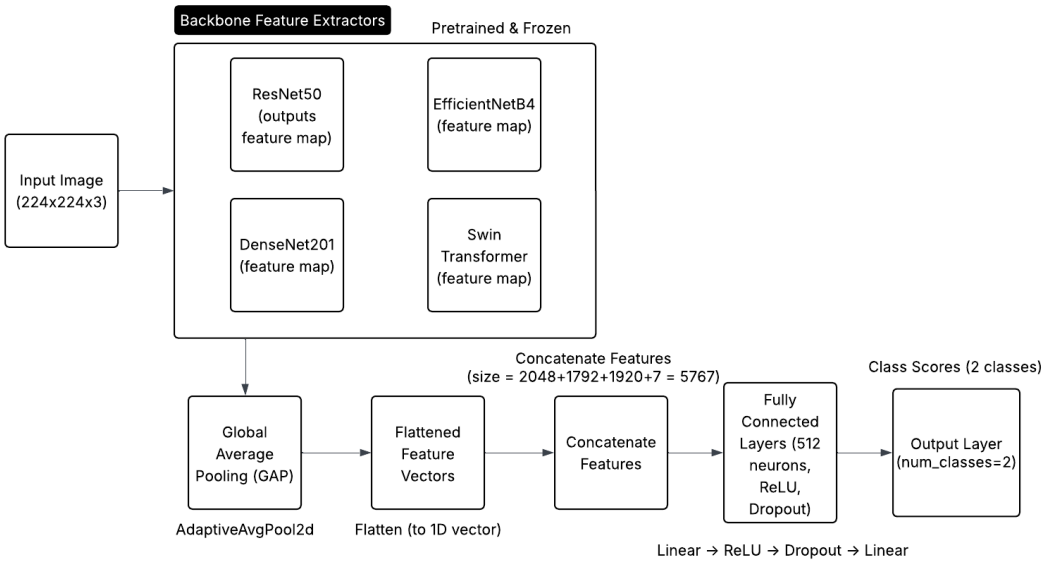

Figure 3.4: General structure of the proposed model and its pipeline

3.8 TRAINING CONFIGURATION AND IMPLEMENTATION DETAILS

To guarantee stability and accuracy, the Berserker model needed to be trained using carefully chosen hyperparameters and training techniques. For faster computation, the training was carried out in Google Colab with GPU acceleration. PyTorch was used to script the training procedure, and a typical supervised learning setup was used to train the model.

The loss function used was CrossEntropyLoss, which is suited well for binary classification problems. To optimize the model weights, the AdamW optimizer was chosen. This optimizer is effective for deep networks and includes weight decay, which helps preventing overfitting. ReduceLROnPlateau, which is a learning rate scheduler, was used to reduce the learning rate automatically when validation performance stopped improving.

During training, mixed precision was enabled using autocast and GradScaler of PyTorch. This allowed faster training with lower memory usage, which is essential

when working with large hybrid models, especially on limited hardware. The model was trained for a fixed number of epochs; wich was 10 epochs for this thesis, depending on early validation results.

Model checkpoints were saved during training using torch.save(). The best model, based on validation F1-score, was saved to Google Drive at:

/content/drive/MyDrive/THESIS/Codes/Custom/custom_best_19and20_train.pth.
This allowed easy reloading and reuse during evaluation or testing.

Additional training techniques included dropout in the classification head to prevent overfitting and batch normalization to stabilize learning. All training and evaluation scripts were kept modular and making it easy to adjust settings or test individual backbones. All in all, the training configuration was chosen to balance speed, accuracy, and reproducibility in a cloud-based environment.

3.9 EXPERIMENTAL SETUPS AND COMPARATIVE MODELS

A number of experiments have been done in order to assess the performance of the Berserker model. Training Berserker on the combined ISIC 2019 and ISIC 2020 training sets and testing it on the official ISIC 2019 test set formed the main experiment. This configuration is regarded as the main experiment and represents the central pipeline of this thesis.

In addition to the main setup, comparative experiments were conducted to explore different training sources and model architectures. First, Berserker was trained only on ISIC 2019 data and tested on the ISIC 2019 test set. Then, Berserker was trained only on 85% of the ISIC 2020 training set and tested on the other 15% as testing set, just because the official testing set of this dataset did not have the groundtruth for its metadata. All the images were picked randomly. These variations allowed for analyzing the effect of dataset size and diversity.

Another group of experiments tested each of the four backbones, ResNet50, EfficientNetB4, DenseNet201 and Swin Transformer, as standalone models. Each backbone was trained individually on ISIC 2019 and ISIC 2020 training sets combined and tested on ISIC 2019 testing set.

Obviously, this helped compare the hybrid architecture to single-model approaches and showed whether Berserker's feature fusion strategy improved results.

In all experiments, except the one that was applied only on ISIC 2020, the test set remained the same: the ISIC 2019 test set with ground truth labels. This consistent testing strategy ensured fair comparison between models. In addition, a threshold tuning process was applied to predictions of each model in order to select the best cutoff for classification. Performance was evaluated using accuracy, sensitivity, precision, specificity, F1-score, and ROC AUC. Accuracy is basically the fraction of all predictions the model got right, regardless of class, while precision shows how trustworthy the melanoma calls are. Recall states how many actual melanomas the model managed to catch, while specificity shows how well you clear benign cases. It is the flip-side of sensitivity. Finally, F1-score is the harmonic mean of precision and recall, a single number that balances false alarms (precision) and missed cases (recall). Particularly handy when classes are imbalanced (like melanoma vs. benign). All of the formulas are shown below and each definition explained in the **Table 3.1**:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{3.1}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{3.2}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{3.3}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} \tag{3.4}$$

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3.5)$$

Table 3.1: Explanation of the formulas' definitions

Symbol	Meaning in this experiment (melanoma = positive)
TP	True Positives (melanoma images correctly labeled melanoma)
TN	True Negatives (benign images correctly labeled benign)
FP	False Positives (benign images incorrectly labeled melanoma)
FN	False Negatives (melanoma images incorrectly labeled benign)

Every experiment used the same pipeline for evaluating each model. Following training, a predetermined threshold was used to compare the model predictions to the ground truth labels. **Algorithm 3.1** explains the entire testing process, and **Algorithm 3.2** explains how performance metrics are calculated. To guarantee fair and repeatable comparisons, these procedures were applied in every evaluation setup.

Algorithm 3.1: Pseudocode for testing the Berserker model

Function predict_label(image I)

Input: Preprocessed skin lesion image III

Output: Predicted label (Melanoma / Benign)

1. Load frozen pre-trained backbones:

    a. ResNet50

    b. EfficientNetB4

c. DenseNet201

d. Swin Transformer

2. Try:

a. Pass image III through each backbone:

i. F1←ResNet50(I)F_1 \leftarrow \text{ResNet50}(I)F1←ResNet50(I)

ii. F2←EfficientNetB4(I)F_2 \leftarrow \text{EfficientNetB4}(I)F2←EfficientNetB4(I)

iii. F3←DenseNet201(I)F_3 \leftarrow \text{DenseNet201}(I)F3←DenseNet201(I)

iv. F4←SwinTransformer(I)F_4 \leftarrow \text{SwinTransformer}(I)F4←SwinTransformer(I)

b. Concatenate features:

Ffused←Concatenate(F1,F2,F3,F4)F_{\text{fused}} \leftarrow \text{Concatenate}(F_1, F_2, F_3, F_4)Ffused←Concatenate(F1,F2,F3,F4)

c. Apply regularization:

i. Fnorm←BatchNormalization(Ffused)F_{\text{norm}} \leftarrow \text{BatchNormalization}(F_{\text{fused}})Fnorm←BatchNormalization(Ffused)

ii. Fdrop←Dropout(Fnorm)F_{\text{drop}} \leftarrow \text{Dropout}(F_{\text{norm}})Fdrop←Dropout(Fnorm)

d. Feed into classification head:

Output←FNN(Fdrop)\text{Output} \leftarrow \text{FNN}(F_{\text{drop}})Output←FNN(Fdrop)

e. Apply activation function:

Probabilities←Softmax(Output)\text{Probabilities} \leftarrow \text{Softmax}(\text{Output})Probabilities←Softmax(Output) or Sigmoid(Output)\text{Sigmoid}(\text{Output})Sigmoid(Output)

f. Determine final label:

Label←Argmax(Probabilities)\text{Label} \leftarrow \text{Argmax}(\text{Probabilities})Label←Argmax(Probabilities) or threshold-based decision

g. Return Label

3. Except ValueError, RuntimeError, MemoryError as e:

a. Log “Error during inference: ” + e

b. Return “Error: Prediction Failed due to input/memory/model issue”

4. End

Algorithm 3.2: Pseudocode for computing classification metrics

---

Function compute_metrics(all_labels, all_predictions, all_probabilities)
Input:

- all_labels: Ground truth labels
- all_predictions: Model-predicted labels

- all_probabilities: Melanoma class probabilities

Output: Accuracy, Precision, Recall, F1 Score, Confusion Matrix, AUC

1. Try:
    a. if number of classes = 2 then
        i. average_type ← 'binary'
    b. else
        i. average_type ← 'weighted'
2. Calculate:
    a. accuracy ← CorrectPredictions / TotalSamples
    b. precision ← TP / (TP + FP)
    c. recall ← TP / (TP + FN)
    d. F1 ← 2 × (precision × recall) / (precision + recall)
    e. confusion_matrix ← [[TN, FP], [FN, TP]]
3. if average_type = 'binary' then
    a. AUC ← Compute AUC using all_labels and all_probabilities
    b. FPR, TPR ← Compute ROC curve
4. Return accuracy, precision, recall, F1, confusion_matrix, AUC
5. Except ValueError, ZeroDivisionError as e:
    a. Log “Error during metric computation: ” + e
    b. Return “Error: Metric Calculation Failed due to invalid input or division by zero”
6. End

These algorithms represent the backbone of the evaluation logic and were applied to every experiment discussed in this thesis. These experiments provided insights into how training data and model architecture affect performance in melanoma classification.

3.10 IMPLEMENTATION ENVIRONMENT AND TOOLCHAIN

All parts of this research, from data preprocessing to model evaluation, were implemented using Python in the Google Colab environment. Google Colab provides access to GPU acceleration, which is crucial for training deep learning models efficiently. All notebooks were developed and executed in this cloud platform.

The main deep learning framework used was PyTorch, chosen for its flexibility and strong community support.

Additional libraries included:

- timm and transformers: for loading pre-trained models like EfficientNet, ResNet, and Swin Transformer.
- albumentations: for applying image augmentations during preprocessing and loading.
- opencv-python, PIL, and os: for image reading and folder management.
- scikit-learn, matplotlib, and seaborn: for evaluation metrics and visualizations.

All datasets were stored in Google Drive, and all Colab notebooks were linked to these folders. This setup made it easy to load, preprocess, and store large datasets during model development.

The code was written in a modular structure across several notebooks:

1. Preprocessing and augmentation
2. Model architecture and training
3. Evaluation and testing

Model weights, logs, and outputs were saved regularly to ensure nothing was lost during Colab disconnects. More importantly, training could be resumed from the last saved checkpoint.

This toolchain supported efficient development, experimentation and tracking of results. It also ensures reproducibility, which is essential for thesis and academic research.

CHAPTER FOUR

RESULTS AND DISCUSSION

4.1 OVERVIEW OF EXPERIMENTAL DESIGN

The results of every experiment conducted with the proposed Berserker model and its comparative baselines are shown and discussed in this chapter. With dermoscopic images from the ISIC 2019 and ISIC 2020 datasets, the experiments aimed to assess performance of the model in melanoma detection. The performance was measured using several evaluation metrics, including accuracy, precision; recall (sensitivity), F1-score, and specificity. These metrics provide a complete view of how the model performs in both detecting melanoma and avoiding false positives.

The main experiment involved training the Berserker model on a balanced combination of ISIC 2019 and ISIC 2020 training data and testing it on the official ISIC 2019 test set, which includes ground truth labels. This setup was used as the primary benchmark for evaluating the model's effectiveness. In addition to this, several comparative experiments were conducted to test the model under different training conditions. These include training Berserker on only ISIC 2019, only ISIC 2020, and training individual backbones (ResNet50, EfficientNetB4, DenseNet201 and Swin Transformer) separately. All results were compared using the same ISIC 2019 test set to ensure fairness.

Each model was evaluated using both fixed and optimized thresholds. A precision-recall vs. threshold plot was used to determine the threshold that maximized the F1-score, which is particularly useful for imbalanced datasets. Test-time augmentation (TTA) was also applied in some experiments to check if robustness could be improved during inference.

The following sections present the detailed results for each model configuration, followed by a discussion of the key findings, comparisons and limitations.

4.2 EVALUATION METRICS AND PERFORMANCE CRITERIA

To measure the performance of the models in detecting melanoma, several standard evaluation metrics were used. These metrics give a balanced understanding of the model's ability to classify both melanoma and benign cases. Since medical classification tasks often involve class imbalance, focusing only on accuracy is not enough. Therefore, this study includes the following metrics: accuracy, precision, recall (sensitivity), specificity, F1-score and the area under the receiver operating characteristic curve (ROC AUC):

- Accuracy: By displaying the proportion of accurate predictions among all predictions, it demonstrates the overall correctness of the model.
- Precision: It indicates the proportion of images that are truly melanoma.
- Recall (Sensitivity): The model's recall determines how well it recognizes real melanoma cases, which is crucial for medical diagnosis.
- Specificity is the ability to correctly identify benign cases, helping avoid unnecessary treatment or fear.
- F1-score: It is helpful for unbalanced datasets because it integrates precision and recall into a single score.
- ROC AUC: It evaluates the model's ability to separate the two classes across all possible thresholds.

In addition to these, confusion matrices were created to show how many true positives, true negatives, false positives, and false negatives were present in each model's output. This helps understand where the model is making errors.

These metrics were computed on the ISIC 2019 test set using the best-performing model checkpoint from training. The next sections show how each model performed based on these metrics, starting with threshold optimization.

4.3 THRESHOLD OPTIMIZATION ANALYSIS

In binary classification, the model typically outputs a probability between 0 and 1. A threshold is then applied to convert this probability into a class label. Normally the default threshold is often 0.5, but this may not always be what is needed, especially in medical datasets that are not balanced. In this research, different thresholds were tested to find the value that maximized the F1-score.

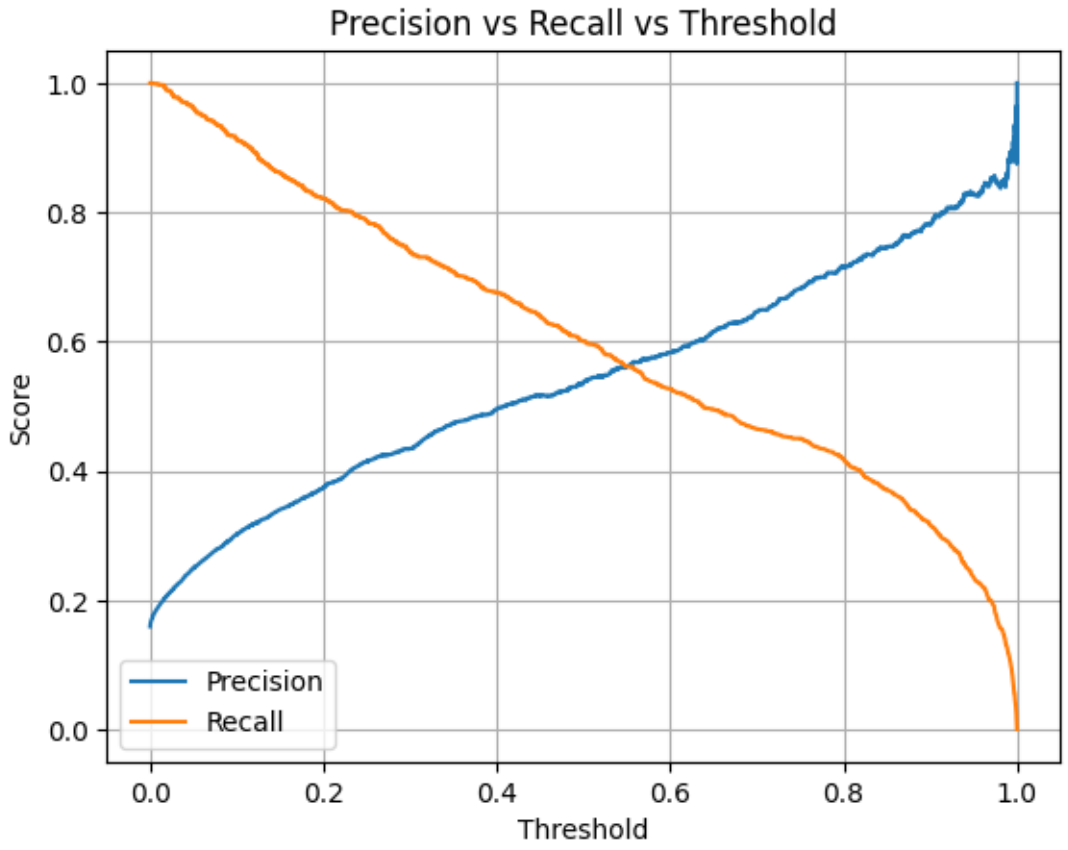

Figure 4.0.1: Precision and recall versus threshold, when the model was trained on both ISIC 2019/2020 and tested only on ISIC 2019 testing set

To identify the best threshold, a precision-recall vs. threshold curve was created for each model. The **Figure 4.1** shows how precision and recall change as the threshold varies. The point where the F1-score reaches its maximum is selected as the optimal

threshold. This approach does help to balance between false positives and false negatives, which is important in melanoma detection where missing a positive case can be serious.

For the main Berserker model trained on the combined ISIC 2019 and ISIC 2020 data, the optimal threshold was found to be 0.43, which performed better than the default value, which is 0.5. Additional thresholds such as 0.47 and 0.6 were also tested in other experiments for comparison. Also the found threshold for the train and test only on ISIC 2019 was 0.47, which is shown in the **Figure 4.2** below.

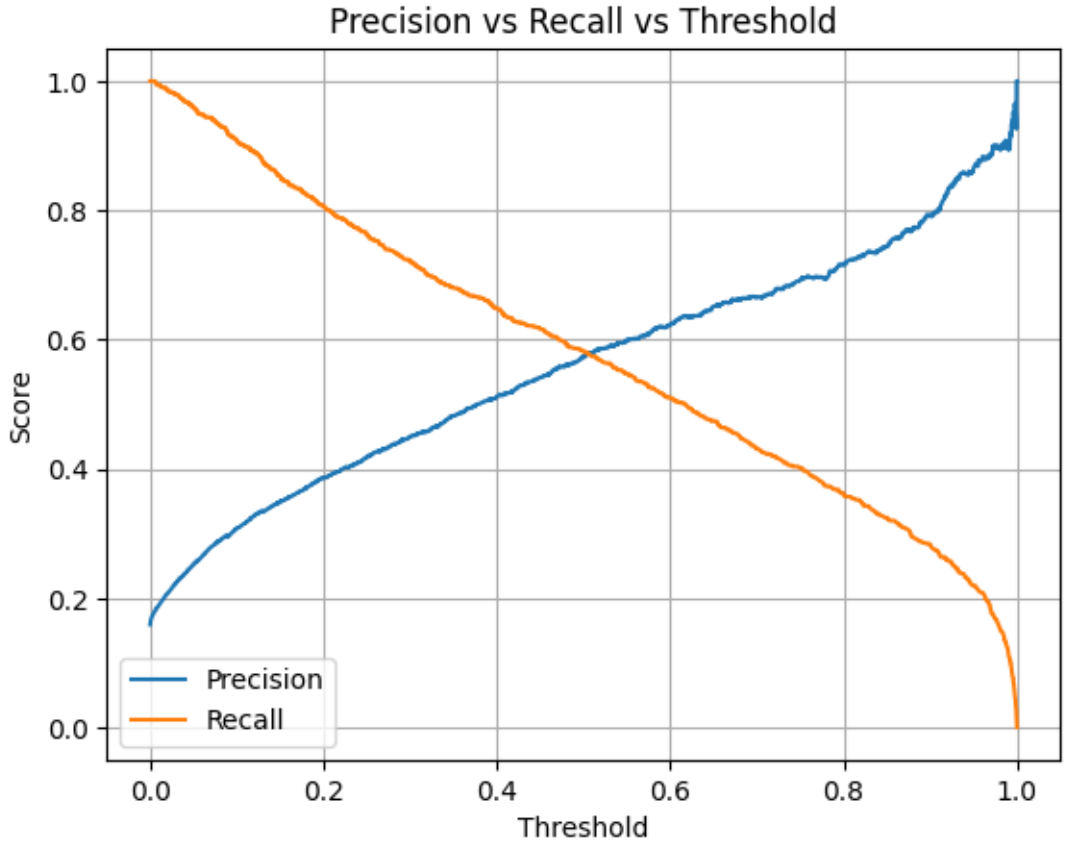

Figure 4.0.2: Precision and recall versus threshold, when the model was trained on only ISIC 2019 training set and tested on ISIC 2019 testing set

Using a tuned threshold helped improve the overall balance between sensitivity and specificity. It also increased the reliability of results when comparing different models and training conditions. In the results tables and figures, both fixed (0.5) and optimized thresholds are reported to give a full picture of model behavior.

This threshold-tuning step is crucial for medical applications where the cost of errors is high. The following sections present the model results with and without threshold optimization.

4.4 RESULTS OF BERSERKER ON ISIC 2019 ONLY

In this experiment, the Berserker model was trained only on the ISIC 2019 training set and evaluated on the official ISIC 2019 test set. The objective was to create a measure of performance when utilizing a single dataset without merging it with ISIC 2020.

To address the obvious class imbalance, intended augmentation was applied to the training set's melanoma cases just before training. To purposefully boost and increase the number of melanoma images and align the class count with benign samples, some methods like flipping, rotation and brightness adjustment were employed. As a result, the model was trained on a dataset that was balanced and had more variability in the melanoma class.

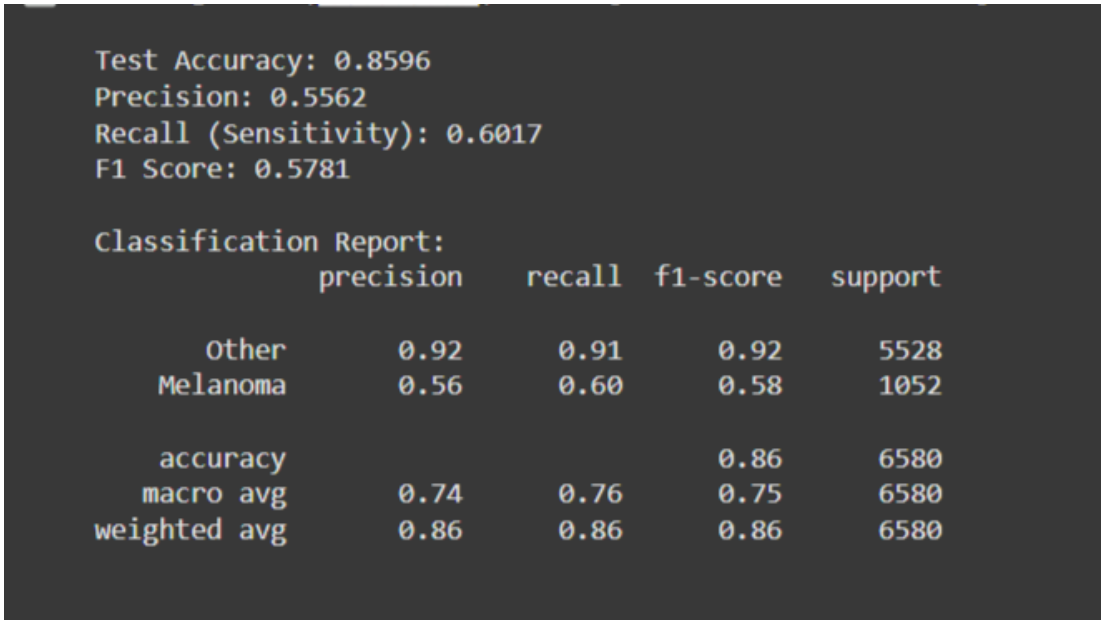

Figure 4.0.3: Evaluation metrics of the Berserker model tested on the ISIC 2019 test set

After training, as it is shown in the **Figure 4.3**, the model showed acceptable performance, achieving an accuracy of 85.96%, precision of 55.62%, sensitivity (recall) of 60.17%, and an F1-score of 57.81%. The classification report indicated that the model detected benign (Other) cases with high precision (0.92) and recall (0.91), while melanoma cases had lower precision (0.56) and recall (0.60).Despite the applied augmentation, the limited diversity of ISIC 2019 compared to a multi-source dataset may have just affected generalization.

On the other hand, the F1-score, which balances precision and recall, was lower compared to the experiment that used both ISIC 2019 and ISIC 2020 for training. These results lend credence to the idea that adding more diverse and complementary data improves the model's ability to detect melanoma.

This experiment shows that while Berserker performs admirably when using ISIC 2019 alone, it can achieve its maximum potential when trained on a more extensive, multi-source dataset.

4.5 RESULTS OF BERSERKER ON ISIC 2020 ONLY

In this experiment, the Berserker model was trained and evaluated using only the ISIC 2020 dataset. The dataset originally contained 33,126 dermoscopic images, but it was heavily imbalanced, with only 584 melanoma cases compared to tens of thousands of benign samples. To address this issue, targeted augmentation was applied only to the melanoma class. Through techniques such as horizontal and vertical flipping, rotation and brightness adjustments, the number of melanoma images was synthetically increased until it matched the benign class. As the result, the image counts for train became 22779 images for benign and 8584 images for melanoma, 4881 benign images and 1839 melanoma images for validation set, then 4882 benign images and 1841 melanoma images for test.

It means after preprocessing and augmentation, the dataset was randomly split into three parts, 70% for training, 15% for validation and 15% for testing. All splits were created from within the same dataset, and no external data was used. Importantly, test-time augmentation (TTA) was not applied in this experiment. Despite that, the model achieved excellent results when tested on the ISIC 2020 test split.

The results on the ISIC 2020 test set were surprisingly strong. The model achieved an overall accuracy of 98.57%, with a precision of 0.9960 and a recall (sensitivity) of 0.9517. The F1-score, which balances both precision and recall, reached 0.9733. All of these results are shown in the **Figure 4.4** as well.

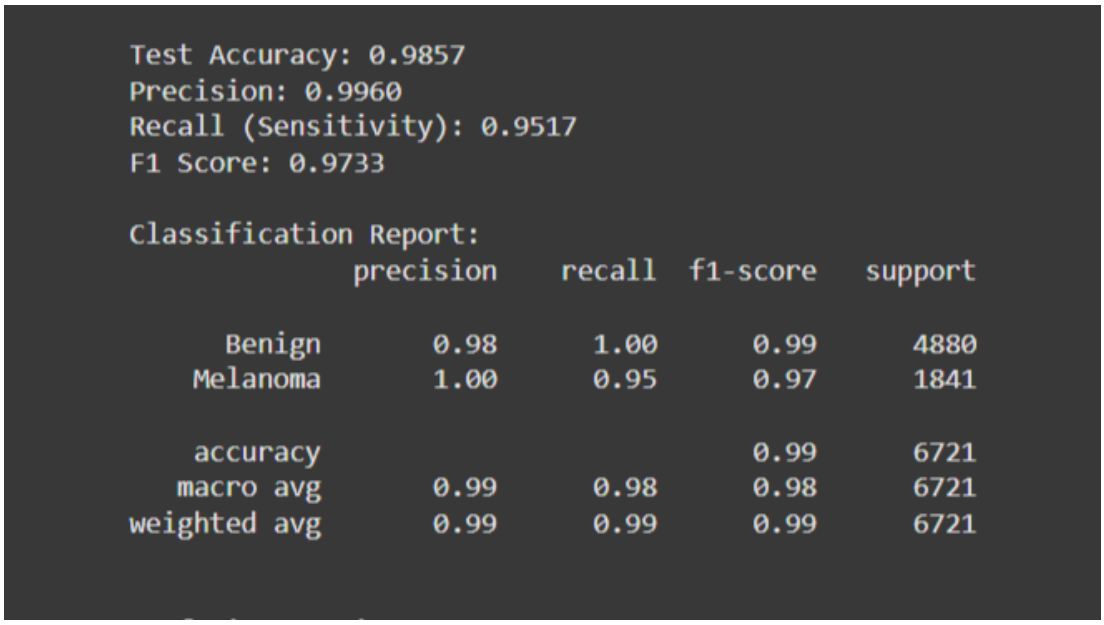

Figure 4.0.4: Evaluation metrics of the Berserker model tested on the custom ISIC 2020 testing set

The confusion matrix, which is also shown in the **Figure 4.5**, confirmed that the model correctly identified the vast majority of melanoma cases, while making very few false positives.

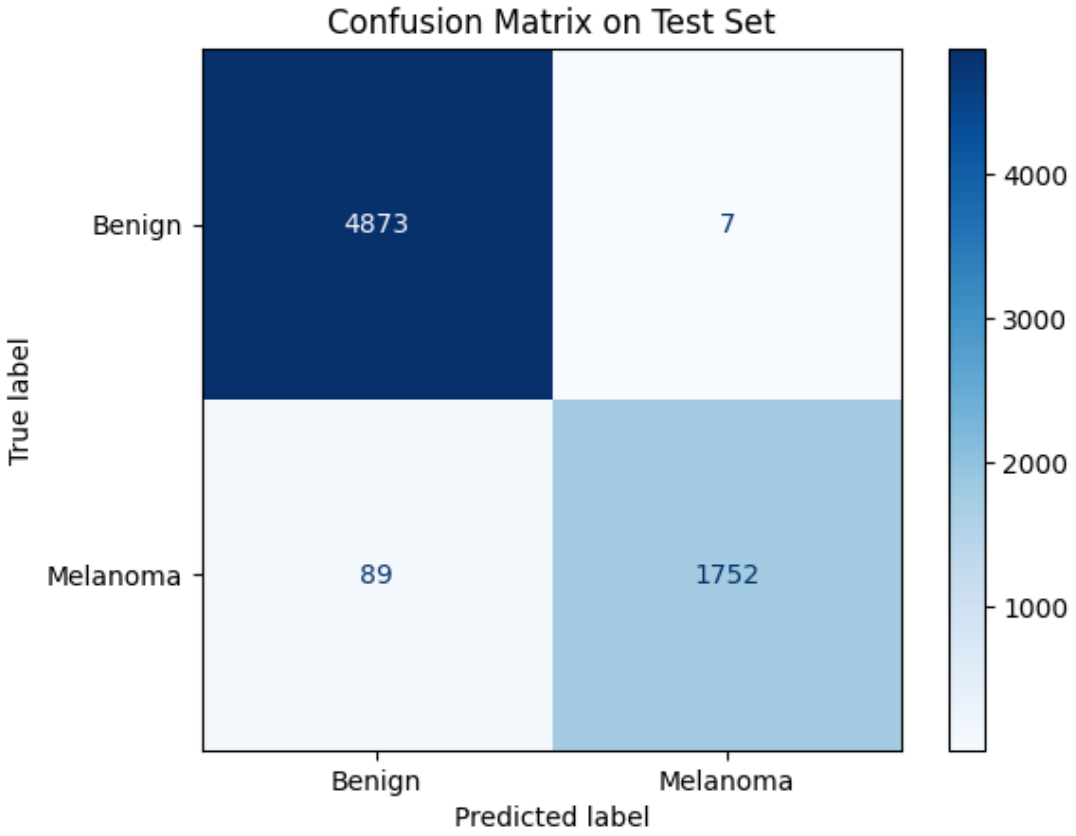

Figure 4.0.5: Model performance visualized with a confusion matrix for ISIC 2020

The results show that the model was quite successful in detecting benign instances as well as melanoma. The accuracy verifies a low false positive rate, and the high recall shows that the model identified almost all melanoma cases.

The uniformity throughout the dataset is probably the reason for the good performance. The model did not experience domain shift because the training and testing pictures were from the same source and had identical properties. This configuration demonstrates how a well-balanced and neatly divided dataset may provide a solid basis for effective melanoma classification without the requirement for TTA.

4.6 RESULTS OF BERSERKER ON COMBINED ISIC 2019 WITH ISIC 2020

This experiment involved training the Berserker model on a combination of ISIC 2019 and ISIC 2020 data. Both datasets were preprocessed and balanced, with 20,809 melanoma and 20,809 benign samples used for training in total at the end. The combined dataset was split into 85% for training and 15% for validation. The evaluation was conducted using the official ISIC 2019 test set, which includes labeled ground truth and had not been seen during training. To improve robustness during testing, test-time augmentation (TTA) was applied, including predictions from the original, horizontally flipped, and vertically flipped versions of each test image. The results of the testing without applying TTA are shown in the **Figure 4.6** and **Figure 4.7** below. In addition, the confusion matrix is shown in the **Figure 4.8**.

```
Testing: 100%|██████████| 52/52 [02:21<00:00,  2.72s/it]

Test Accuracy: 0.8444
Precision: 0.5104
Recall (Sensitivity): 0.6540
F1 Score: 0.5733

Classification Report:
              precision    recall  f1-score   support

      Benign       0.93      0.88      0.90      5528
    Melanoma       0.51      0.65      0.57      1052

    accuracy                           0.84      6580
   macro avg       0.72      0.77      0.74      6580
weighted avg       0.86      0.84      0.85      6580
```

Figure 4.6: Evaluation metrics of the Berserker model trained on ISIC 2019/2020 training set and tested on the ISIC 2019 test set (without TTA)

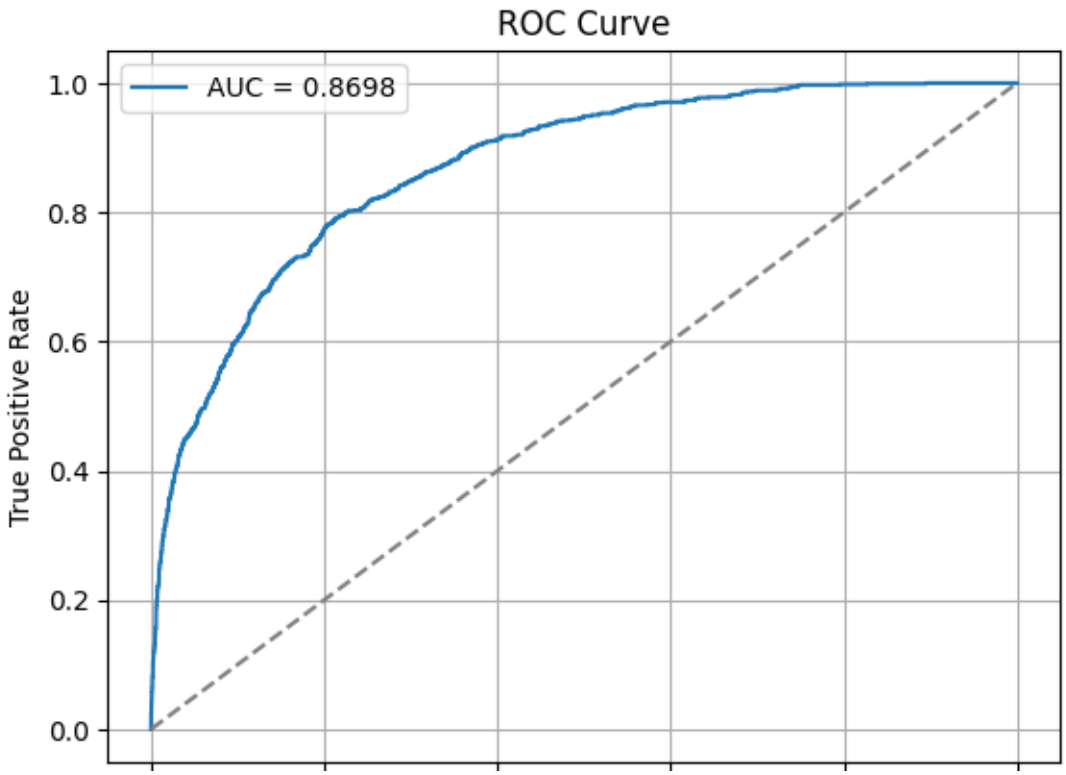

Figure 4.7: Receiver Operating Characteristic (ROC) curve without applying TTA

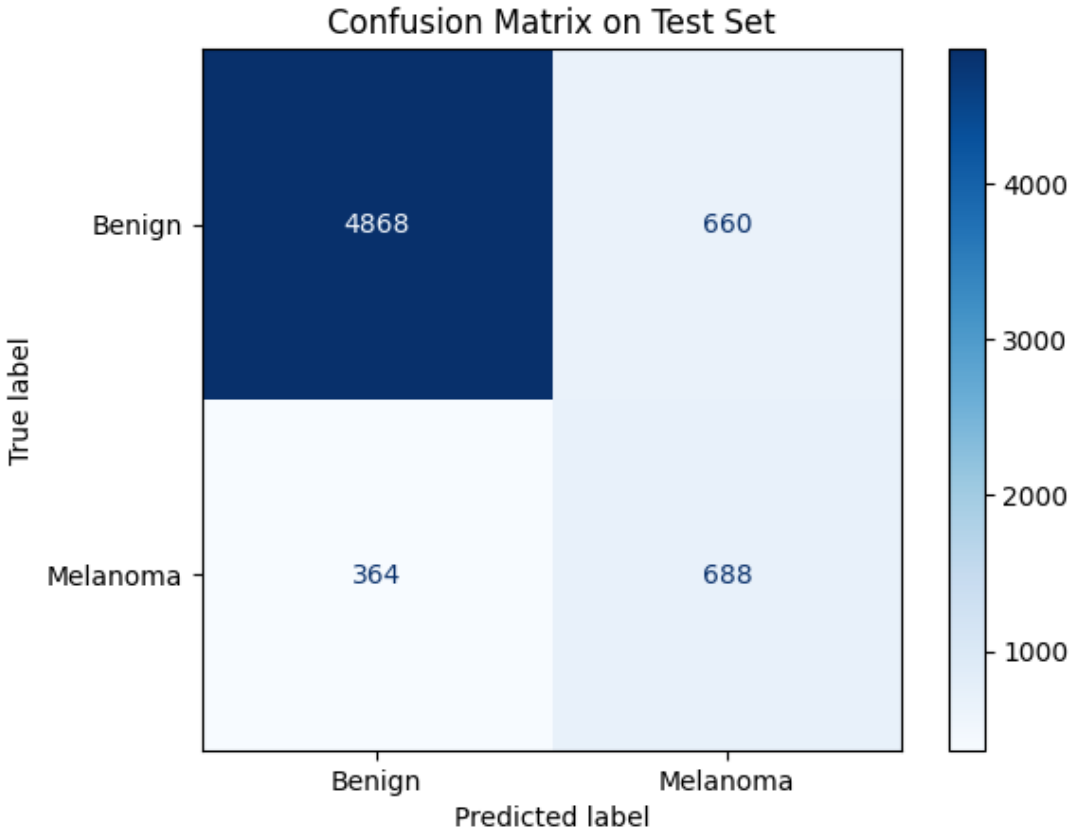

Figure 4.8: Model performance visualized with a confusion matrix for ISIC 2019 testing set while trained on ISIC 2019/2020 (without TTA)

Despite using more data and applying TTA, the model's performance was noticeably lower compared to the ISIC 2020 only setup. The results on the ISIC 2019 test set after applying TTA, which are also shown in the **Figure 4.9** and **Figure 4.10**, with the confusion matrix shown in **Figure 4.11**, were as follows:

- Accuracy: 84.95%
- Precision: 0.5234
- Recall (Sensitivity): 0.6597
- F1 Score: 0.5837
- AUC: 0.8800

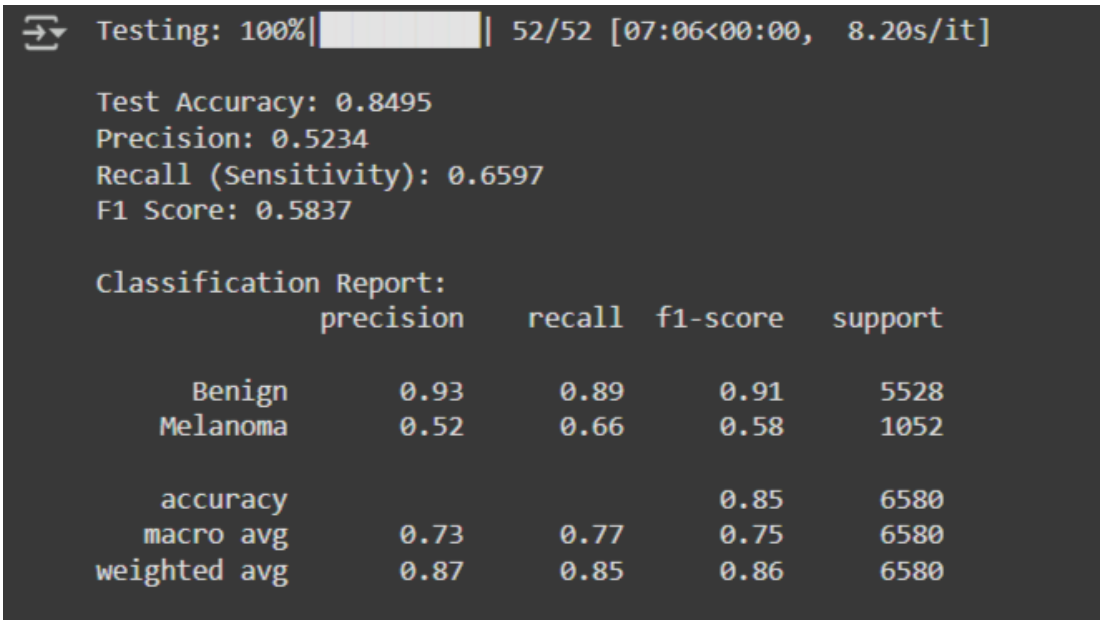

Figure 4.9: Evaluation metrics of the Berserker model trained on ISIC 2019/2020 training set and tested on the ISIC 2019 test set (with TTA)

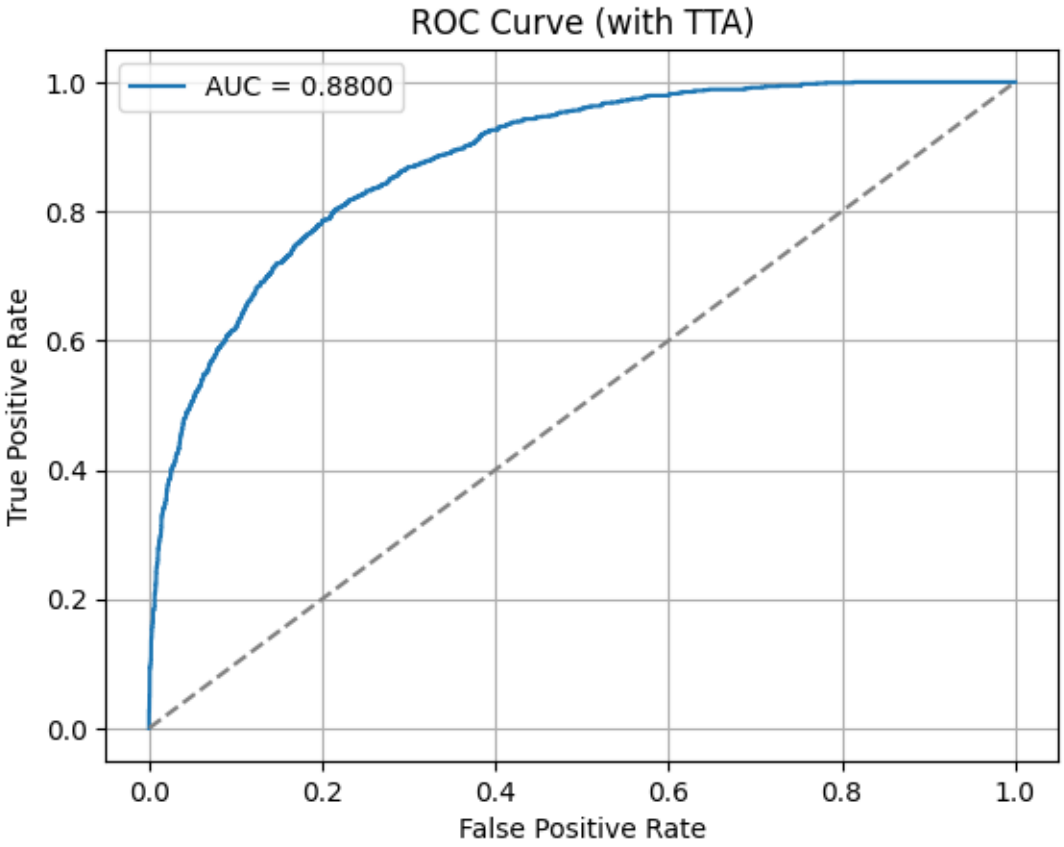

Figure 4.10: Receiver Operating Characteristic (ROC) curve after applying TTA

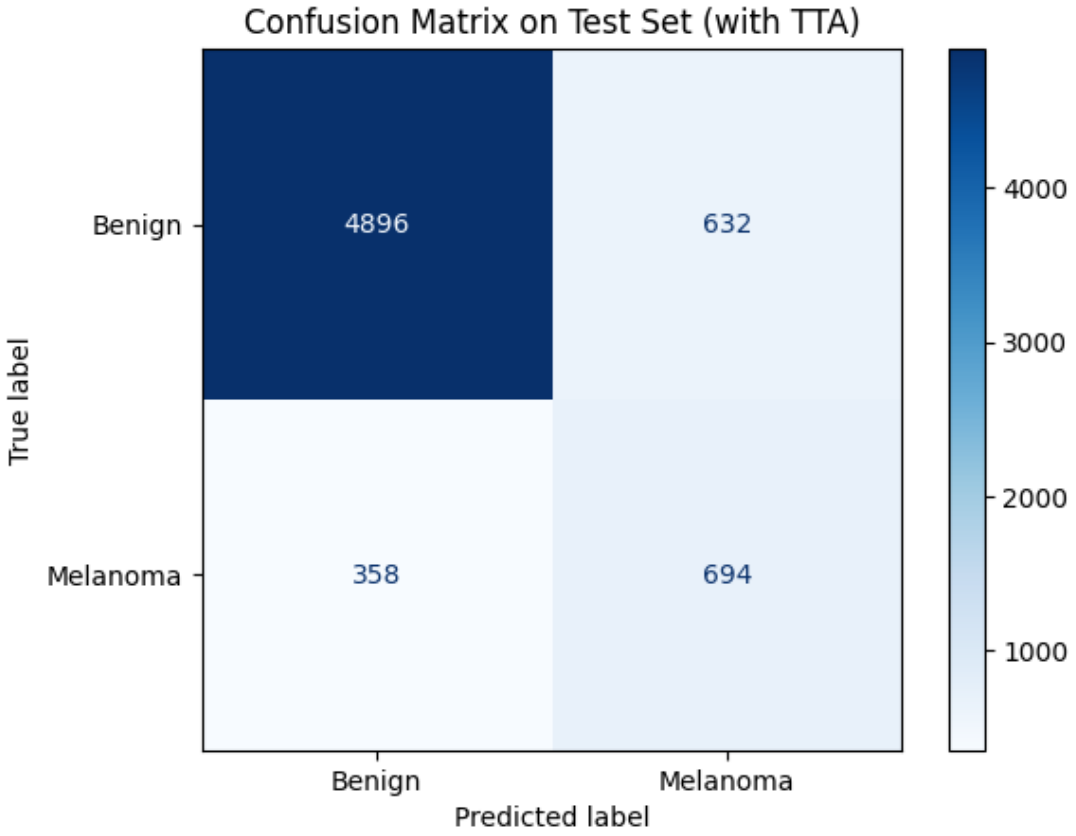

Figure 4.11: Model performance visualized with a confusion matrix for ISIC 2019 testing set while trained on ISIC 2019/2020 (with TTA)

Although the recall was relatively acceptable, the precision and F1-score dropped significantly. The confusion matrix showed a large number of false positives and moderate false negatives. This suggests that while the model could detect melanoma cases, it struggled to distinguish them confidently from benign ones.

A likely reason for the reduced performance is domain shift between the two datasets. Differences in image resolution, lighting, annotation style, and device types may have introduced inconsistencies. Even though the model had access to more images and benefited from TTA, the variation between ISIC 2019 and 2020 may have reduced its ability to generalize effectively to the ISIC 2019 test set.

This result highlights that dataset compatibility and consistency are just as important as dataset size. Simply combining more data from different sources does not always lead to better outcomes, especially in sensitive medical classification tasks.

4.7 RESULTS OF INDIVIDUAL BACKBONES

To better understand the contribution of each component in the Berserker model, all four backbone networks were evaluated separately. These include ResNet50, EfficientNetB4, DenseNet201, and Swin Transformer. Each model was trained individually using the combined ISIC 2019 and ISIC 2020 training datasets, with both classes, melanoma and benign, balanced to 20,809 images each through preprocessing and augmentation.

All backbones followed the same pipeline in terms of data loading, normalization, and image resizing. After training, each model was evaluated on the ISIC 2019 test set, which includes labeled ground truth. To improve prediction stability and handle orientation variability, test-time augmentation (TTA) was used during testing. This involved averaging predictions from the original, horizontally flipped, and vertically flipped versions of each test image.

ResNet50 performed the strongest among all the standalone models. It reached an accuracy of 87.33%, with a recall of 71.01% and an F1-score of 0.6418. The AUC score was 0.9005, which indicates strong discriminative power. The results suggest that ResNet50 is quite balanced; it catches most melanoma cases while maintaining a fair level of precision.

EfficientNetB4 gave slightly lower results, with an accuracy of 86.49% and an F1-score of 0.6015. While it performed reasonably well overall, it was slightly more conservative in identifying melanoma. The recall dropped to 63.78%, and precision also fell a bit compared to ResNet50. Still, its performance was stable and consistent across the test set.

Interestingly, DenseNet201 delivered the highest sensitivity, with a recall of 74.14%, which means it was better than the others at detecting melanoma. However, this came with a cost; it had more false positives, which lowered its precision to 53.68%. The overall F1-score was 0.6228, and the AUC was a strong 0.9014. This model was more aggressive, favoring detection over caution.

Swin Transformer, a vision transformer-based model, had mixed results. It achieved a recall of 72.05%, which was comparable to DenseNet201, but its precision was the lowest among all four models at 50.50%. The F1-score stood at 0.5938, and accuracy dropped slightly to 84.24%. It seems that while Swin Transformer is sensitive to melanoma cases, it also misclassified a larger number of benign images.

This experiment demonstrated the unique strengths of each backbone. DenseNet201 dominated in sensitivity, while ResNet50 provided the best balance between recall and precision. Swin Transformer had a high recall rate as well, but it was more likely to cause false alarms. The fundamental tenet of the Berserker model is supported by these findings: combining various feature extractors lessens their individual flaws and yields more reliable results. These backbones work well together and offer more comprehensive feature representations for the final classification. In Addition, all four backbones were compared fully in the **Table 4.1** below.

Table 4.1: Comparative performance of deep learning backbones

Metric	RESNET	EFFICIENTNET	DENSENET	SWIN
Test Accuracy	0.8698	0.8597	0.8529	0.835
Precision	0.5786	0.5551	0.5297	0.4889
Recall (Sensitivity)	0.6825	0.6179	0.7129	0.7091
F1 Score	0.6263	0.5848	0.6078	0.5787
AUC	0.8921	0.8733	0.8905	0.8725
Inference Time (approx.)	4.25 s	0.17 s	0.17 s	0.17 s
Benign Precision	0.94	0.93	0.94	0.94
Benign Recall	0.91	0.91	0.88	0.86

Table 4.1 (Continued): Comparative performance of deep learning backbones

Benign F1-score	0.92	0.92	0.91	0.9
Melanoma Precision	0.58	0.56	0.53	0.49
Melanoma Recall	0.68	0.62	0.71	0.71
Melanoma F1-score	0.63	0.58	0.61	0.58
Benign True Positives	5005	5007	4862	4748
Benign False Positives	523	521	666	780
Melanoma False Negatives	334	402	302	306
Melanoma True Positives	718	650	750	746
Total Support	6580	6580	6580	6580

4.8 IMPACT OF TEST-TIME AUGMENTATION

To better assess how stable the model's predictions were, test-time augmentation (TTA) was applied during the evaluation phase. In this method, each test image was passed through the model multiple times using different transformations, specifically horizontal flips, vertical flips, and the original image. The predicted probabilities from each version were then averaged to produce the final score for that image.

In most cases, TTA led to a small but meaningful improvement in the performance of the model. While overall accuracy and F1-score increased slightly, the most noticeable benefit was in sensitivity. The model became better at identifying difficult melanoma cases, especially those that were borderline or looked slightly different from the training data. This was particularly helpful when the confidence values were near the decision threshold.

Even though the metric gains were not dramatic, the added consistency was valuable. Dermoscopic images can vary in orientation and lighting, and viewing them from different angles, as TTA simulates, helped the model behave more reliably. It is a

simple technique, but in this study, it proved useful, especially when applied to the Berserker model trained on the combined ISIC 2019 and 2020 dataset.

The main drawback is that TTA increases inference time, which might be a limitation in real-time or clinical settings. However, for offline research and testing scenarios like this one, it offered a practical way to strengthen model robustness with minimal extra work.

Overall, TTA turned out to be an effective, low cost strategy that contributed to more reliable melanoma classification, and it is a method worth considering in similar medical imaging tasks.

4.9 COMPARATIVE PERFORMANCE VS EXISTING METHODS

To put the Berserker model's performance into context, its results were compared with existing models reported in recent studies on melanoma classification. While not all setups are directly comparable due to differences in datasets, preprocessing methods and evaluation protocols, it is still useful to highlight where this work stands relative to others in the field.

In previous research, models like ResNet50, MobileNetV2, and VGG16 have been commonly used on the ISIC datasets. Reported F1-scores for these models generally fall between 0.70 and 0.85, with precision and recall varying based on whether data augmentation, balancing, or transfer learning was applied. Some studies also implemented ensemble or voting-based techniques to improve stability, though often at the cost of increased complexity or longer inference times.

In contrast, the Berserker model, when trained on ISIC 2020 alone, achieved a remarkably high F1-score of 0.9733, with accuracy at 98.57% and recall reaching 95.17%. These results surpass most of the previously reported single-model performances. Even when trained on the combined ISIC 2019 and 2020 datasets (with domain shift and added variability), the model still held a respectable F1-score of

0.5837 on the ISIC 2019 test set, despite being affected by the inconsistencies between the datasets.

It is also worth noting that few existing works have explored a hybrid model architecture that combines both CNNs and vision transformers. Most focus on tuning or stacking CNNs. In this thesis, integrating four distinct backbones, which are ResNet50, EfficientNetB4, DenseNet201 and Swin Transformer, offered complementary strengths, contributing to the high performance achieved, particularly on consistent datasets like ISIC 2020.

These findings imply that Berserker offers a flexible and competitive solution, particularly when trained under controlled conditions, even though a one-to-one comparison may not always be feasible. It presents a novel hybrid design that is uncommon in previous melanoma detection work and provides a workable substitute for conventional single-model approaches.

CHAPTER FIVE

CONCLUSION AND FUTURE WORK

5.1 OVERVIEW OF THE RESEARCH OBJECTIVES

This thesis set out to design and evaluate a deep learning-based system for automated melanoma classification using dermoscopic images. The main goal was to explore whether combining multiple neural network backbones into a hybrid architecture could improve classification accuracy, especially in the presence of class imbalance and domain variability. The project used two large publicly available datasets, which were ISIC 2019 and ISIC 2020, as sources of training and testing data. These datasets differ in format and structure, making the task more challenging but also more realistic.

To address these challenges, a custom hybrid model called Berserker was developed. This model integrates four popular and well-known backbones: ResNet50, EfficientNetB4, DenseNet201, and Swin Transformer. The goal was to leverage the different feature extraction strengths of these architectures in a parallel configuration. Additionally, the thesis examined the effects of dataset balancing, augmentation, test-time augmentation, and threshold tuning on classification performance.

Along with developing a model, the study attempted to evaluate the proposed model in various scenarios, such as training on the ISIC 2020 test set, testing on the ISIC 2019 test set, and training on the ISIC 2020 and 2019 test sets. The efficacy of the model was evaluated several important performance metrics including accuracy, F1-score, precision, recall and AUC. The findings of this study gave researchers a better understanding of how preprocessing techniques, model design and data source consistency impact deep learning systems ability to classify medical images.

5.2 SUMMARY OF METHODOLOGICAL CONTRIBUTIONS

The methodology presented in this thesis combines multiple practical strategies to build a high-performing melanoma classification model. First, the datasets were carefully preprocessed. For ISIC 2019, only the melanoma and benign classes were selected, and all images were normalized and resized. The ISIC 2020 dataset required conversion from DICOM to image, normalization and significant augmentation due to extreme class imbalance. In both cases, synthetic augmentation was applied to the melanoma class to create balanced training sets with equal representation for both classes.

The most notable contribution was the design and implementation of the Berserker model, which combines four pretrained backbones into a single hybrid architecture. Each backbone was frozen to reduce training time and overfitting, while the classification head was trained on top of the combined features. This architectural choice was tested alongside traditional approaches using each backbone individually to establish comparative baselines.

Other methodological features include the use of TTA to improve inference robustness and threshold tuning to optimize the decision boundary for binary classification. Additionally, the experiments were run on Google Colab using GPU acceleration, and all training, validation and testing splits were managed through carefully scripted pipelines to ensure reproducibility.

Overall, the methodology combined best practices from deep learning, medical imaging, and experimental design. It demonstrated that hybrid architectures, when trained on well-prepared datasets and evaluated properly, can perform strongly, even under real world constraints like domain shift and limited ground truth availability.

5.3 KEY EXPERIMENTAL FINDINGS

The experiments showed that the Berserker model is capable of achieving strong results in melanoma classification, particularly when trained on the ISIC 2020 dataset alone. In that setting, it reached a high F1-score of 0.9733, with accuracy of 98.57% and sensitivity of 95.17%, without even using test-time augmentation. These results suggest that when the training and testing data come from the same domain and are well-prepared, the hybrid model performs extremely well.

In contrast, training on the combined ISIC 2019 and 2020 datasets, and testing on the ISIC 2019 test set, resulted in a noticeable drop in performance. Despite applying TTA and class balancing, the F1-score dropped to 0.5837 and precision fell significantly. This highlighted how domain shift and inconsistent data quality between datasets can negatively impact generalization.

When evaluating each backbone individually, it became clear that each model had unique strengths. DenseNet201 performed best in sensitivity, ResNet50 offered the most balanced performance, and Swin Transformer had high recall but suffered from low precision. These findings support the hybrid design of Berserker, where the fusion of diverse features leads to more stable performance overall.

5.4 COMPARATIVE ANALYSIS WITH EXISTING METHODS

The performance of the proposed Berserker model was compared with models commonly used in previous melanoma detection research. While exact comparisons are difficult due to different dataset setups and evaluation methods, the results provide a general indication of the model's competitiveness. Models such as ResNet50, MobileNet, and DenseNet are frequently used in literature, with typical F1-scores reported between 0.70 and 0.85 when trained on ISIC datasets.

In contrast, Berserker achieved an F1-score of 0.9733 on the ISIC 2020 test set, which is significantly higher than most reported results in related work. Even under domain

shift conditions, when trained on combined ISIC 2019 and 2020 data and tested on ISIC 2019, the model still performed reasonably, though sensitivity and precision dropped.

What makes Berserker different is its hybrid architecture. Instead of relying on a single CNN, it integrates both CNN and transformer-based models, allowing it to capture a broader range of features. This architectural choice appears to be effective, especially when the training data is consistent and well prepared.

5.5 LIMITATIONS AND CHALLENGES

Although the model performed well overall, several limitations were identified during the research. First, all backbone networks were kept frozen during training. While this helped reduce overfitting and computational cost, it may have limited the ability of the model to learn task-specific patterns in medical images.

Another challenge was the presence of domain shift between ISIC 2019 and ISIC 2020 datasets. Even after balancing and augmentation, differences in image characteristics led to a drop in performance when the model was trained on one dataset and tested on the other.

In addition, no patient metadata was used in this study. Features like age, gender, and lesion location could have helped improve classification, especially in borderline cases. Finally, while test-time augmentation increased sensitivity, it also increased inference time, which may not be suitable for real-time clinical use.

5.6 PRACTICAL IMPLICATIONS FOR MEDICAL AI

Working on this project has shown how powerful deep learning models can be in supporting medical diagnosis, especially when they are trained and evaluated properly. The strong results from the Berserker model, particularly when using the ISIC 2020 dataset alone, suggest that carefully designed hybrid architectures can perform well in real-world tasks like melanoma detection.

In practice, a model like this could be used as a support tool for dermatologists. It would not replace medical expertise, but it could help flag suspicious lesions or speed up screening in high-risk cases. This might be especially useful in clinics where experienced specialists are not always available. Even if a model like Berserker is not perfect, it could reduce workload and help focus attention where it's most needed.

One thing that became clear is the importance of consistent and clean data. When datasets from different sources (ISIC 2019 and 2020) were combined, the performance dropped, even though more training data were used. This showed that having more data is not enough; it also needs to match the target conditions. For medical AI, this is a big deal, because hospitals and clinics often use different equipment and even lighting can affect how lesions look in images.

Overall, the experiments in this thesis show that AI has real potential in the medical field, but it needs to be developed and tested carefully. Building trust in these systems means paying attention not just to accuracy but also to consistency, explainability, and how well the model works across different environments.

5.7 FUTURE WORK AND MODEL IMPROVEMENTS

This work could be extended several ways. One of the most important next steps would be to fine-tune the backbone networks, instead of freezing them. This could allow the model for better adaption to medical image features and improve generalization.

Another possible improvement is of course to explore feature fusion techniques more deeply. In this thesis, the fusion method was relatively straightforward. Trying attention-based or adaptive fusion layers could enhance performance.

It would also be valuable to train and test the model on clinical photographs or non-dermoscopic data to assess its flexibility. Adding patient metadata as an additional

input channel is another potential improvement, as this information could help the model to make more informed predictions.

Lastly, deploying the model in a real-time setting would require optimizing inference time and testing usability in clinical workflows.

5.8 FINAL REMARKS AND THESIS SUMMARY

This thesis introduced Berserker, a custom hybrid deep learning model designed to detect melanoma in dermoscopic images. Through extensive experiments on ISIC 2019 and ISIC 2020 datasets, it was shown that the model performs strongly when trained on consistent and balanced data. The project also explored how dataset diversity, architecture design, and augmentation strategies affect performance.

Although there were challenges, such as domain shift and the limitations of frozen backbones, the research provided valuable insights into how hybrid architectures can support medical AI tasks. The results indicate that combining multiple feature extractors can improve reliability and classification strength, especially in sensitive applications like cancer detection.

In summary, this work contributes a tested and reproducible approach to deep learning in dermatology and opens several pathways for future development and clinical integration.

REFERENCES

Abdulredah, A. A., Fadhel, M. A., Alzubaidi, L., Duan, Y., Kherallah, M. and Charfi, F. (2025). Towards Unbiased Skin Cancer Classification Using Deep Feature Fusion. *BMC Medical Informatics and Decision Making*, 25, Article 89. [online] Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC11786435 [Accessed 8 Jul. 2025].

Akter, M., Khatun, R., Talukder, M. A., Islam, M., Uddin, M. A., Ahamed, M. K. U. and Khraisat, A. (2025). An Integrated Deep Learning Model for Skin Cancer Detection Using Hybrid Feature Fusion Technique. [online] Available at: https://www.researchgate.net/publication/387870775 [Accessed 8 Jul. 2025].

Albahli, S. (2025). A Robust YOLOv8-Based Framework for Real-Time Melanoma Detection and Segmentation with Multi-Dataset Training. *Diagnostics (Basel)*, 15(6), 691. [online] Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC11941149 [Accessed 8 Jul. 2025].

Ali, R. and Ragb, H. K. (2021). Skin Lesion Segmentation and Classification Using Deep Learning and Handcrafted Features. [online] Available at: https://www.bohrium.com/paper/read?libraryId=6209507 [Accessed 8 Jul. 2025].

el Mrabet, A., Mohamed, B., Alihamidi, I., Kouach, B., Hlou, L. and El Gouri, R. (2025). Enhancing Early Detection of Skin Cancer in Clinical Practice with Hybrid Deep Learning Models. [online] Available at: https://www.researchgate.net/publication/390493428 [Accessed 8 Jul. 2025].

Grignaffini, F., Troiano, M., Barbuto, F., Simeoni, P., Mangini, F., D'Andrea, G., Piazzo, L., Cantisani, C., Musolff, N., Ricciuti, C. and Frezza, F. (2023). Anomaly detection for skin lesion images using convolutional neural network and injection of handcrafted features: A method that bypasses the preprocessing of dermoscopic images. *Algorithms*, 16(10), 466. [online] Available at: https://doi.org/10.3390/a16100466 [Accessed 17 Jun. 2025].

Hasan, M. Z. and Rifat, F. Y. (2025). Hybrid Ensemble with Segmentation-Assisted Classification and GBDT for Skin Cancer Detection with Engineered Metadata and Synthetic Lesions from ISIC 2024 Non-Dermoscopic 3D-TBP Images. [online] Available at: https://www.researchgate.net/publication/392405954 [Accessed 8 Jul. 2025].

Lilhore, U. K., Sharma, Y. K., Simaiya, S., Alroobaea, R., Baqasah, A. M., Alsafyani, M. and Alhazmi, A. (2025). SkinEHDLF: A Hybrid Deep Learning Approach for Accurate Skin Cancer Classification. *Scientific Reports*, [online] Available

at: https://www.nature.com/articles/s41598-025-98205-7 [Accessed 8 Jul. 2025].

Mateen, M., Hayat, S., Arshad, F., Gu, Y.-H. and Al-antari, M. A. (2024). Hybrid Deep Learning Framework for Melanoma Diagnosis Using Dermoscopic Medical Images. *Diagnostics*, 14(19), 2242. [online] Available at: https://www.mdpi.com/2075-4418/14/19/2242 [Accessed 8 Jul. 2025].

Nandi, S. (2025). New AI Model Sets Benchmark in Skin Cancer Detection with 99.8% Accuracy. [online] Available at: https://www.azorobotics.com/News.aspx?newsID=15937 [Accessed 8 Jul. 2025].

Perera, W. S., Islam, A. R., Vung, P. V. and An, M. (2024). Melanoma Classification Through Deep Ensemble Learning and Explainable AI. [online] Available at: https://www.researchgate.net/publication/378484173 [Accessed 8 Jul. 2025].

Qamar, S., Qadri, S. F., Alroobaea, R., Alshmrani, G. M. M. and Jiang, R. (2025). ScaleFusionNet: Transformer-Guided Multi-Scale Feature Fusion. *arXiv preprint arXiv:2503.03327*. [online] Available at: https://arxiv.org/abs/2503.03327 [Accessed 8 Jul. 2025].

Sancar, Y. (2024). Enhanced Classification of Skin Lesions Using Fine-Tuned MobileNet and DenseNet121 Models. [online] Available at: https://www.researchgate.net/publication/387464772 [Accessed 8 Jul. 2025].

Singh, R. K., Gorantla, R., Allada, S. G. R. and Narra, P. (2022). SkiNet: An Explainable Deep Learning Approach for Real-Time Skin Lesion Detection Using Semantic Segmentation and Uncertainty Estimation. *Computers in Biology and Medicine*, 149, 105948. [online] Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC9621459 [Accessed 8 Jul. 2025].

Ye, Z., Zhang, D., Zhao, Y., Chen, M., Wang, H., Seery, S., Qu, Y., Xue, P. and Jiang, Y. (2024). Deep learning algorithms for melanoma detection using dermoscopic images: A systematic review and meta-analysis. *Artificial Intelligence in Medicine*, 155, p.102934. [online] Available at: https://www.sciencedirect.com/science/article/abs/pii/S0933365724001763?utm_source=chatgpt.com [Accessed 16 Jun. 2025].

Zhang, P. and Chaudhary, D. (2024). Hybrid Deep Learning Framework for Enhanced Melanoma Detection. *arXiv preprint arXiv:2408.00772*. [online] Available at: https://arxiv.org/abs/2408.00772 [Accessed 8 Jul. 2025].

APPENDIX A

LIBRARIES

```
# All of these libraries were used for specific tasks during this thesis for all files
!pip install transformers
!pip install opencv-python
!pip install tensorboard
!pip install timm
import cv2
import os
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torch.cuda.amp import GradScaler, autocast
from torch.utils.tensorboard import SummaryWriter
from torch.optim.lr_scheduler import StepLR
from torchvision import models, transforms
import time
from transformers import CLIPModel, CLIPProcessor, ViTModel,
ViTFeatureExtractor
import numpy as np
import torch.optim as optim
from sklearn.metrics import f1_score, precision_score, recall_score, accuracy_score,
roc_auc_score, confusion_matrix, classification_report
```

```
import seaborn as sns
from PIL import Image
from timm import create_model
from torch.optim.lr_scheduler import ReduceLROnPlateau
from tqdm import tqdm
import shutil
import random
from pathlib import Path
import albumentations as A
import torchvision.transforms as T
from google.colab import drive
drive.mount('/content/drive')
```

APPENDIX B

DATALOADERS

```
# Loading Data

# This is from the code file TrainAndTestISIC2020.ipynb

# UPDATED PATHS

BASE_PATH =
"/content/drive/MyDrive/THESIS/Dataset/kaggle/trainAndTest_from_train"

METADATA_PATH =
"/content/drive/MyDrive/THESIS/Dataset/kaggle/ISIC_2020_Train_Metadata.csv"



TRAIN_PATH = os.path.join(BASE_PATH, "train")

VAL_PATH = os.path.join(BASE_PATH, "val")

TEST_PATH = os.path.join(BASE_PATH, "test")



# Helper to gather image paths and labels

def get_image_paths_and_labels(root_dir):

    image_paths = []

    labels = []

    class_names = sorted(os.listdir(root_dir))  # ['Benign', 'Melanoma']
```

```
    class_to_idx = {cls_name: idx for idx, cls_name in enumerate(class_names)}

    for cls_name in class_names:
        cls_folder = os.path.join(root_dir, cls_name)
        for fname in os.listdir(cls_folder):
            if fname.endswith(('.jpg', '.png')):
                img_path = os.path.join(cls_folder, fname)
                # Optionally check existence here if you want
                image_paths.append(img_path)
                labels.append(class_to_idx[cls_name])
    return image_paths, labels

# Visualize samples function unchanged
def visualize_samples_from_folder(base_path, num_samples=6):
    import random
    samples = []
    for label in ["Benign", "Melanoma"]:
        class_dir = os.path.join(base_path, label)
```

```
        all_files = [os.path.join(class_dir, f) for f in os.listdir(class_dir) if
f.endswith(('.jpg', '.png'))]
        samples.extend(random.sample(all_files, min(len(all_files), num_samples // 2)))

    plt.figure(figsize=(15, 10))
    for idx, img_path in enumerate(samples):
        image = cv2.imread(img_path)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        label = os.path.basename(os.path.dirname(img_path))

        plt.subplot(2, 3, idx + 1)
        plt.imshow(image)
        plt.title(f"Label: {label}")
        plt.axis('off')
    plt.tight_layout()
    plt.show()

print("\nVisualizing sample training images...")
visualize_samples_from_folder(TRAIN_PATH)
```

```
# Dataset class unchanged, but will receive lists, not folder paths
class ISICDataset(Dataset):
    def __init__(self, image_paths, labels, transform=None):
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        # Try loading the image; skip if unreadable
        max_attempts = 10
        attempt = 0
        while attempt < max_attempts:
            img_path = self.image_paths[idx]
            image = cv2.imread(img_path)
```

```
            if image is not None:
                image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
                if self.transform:
                    image = self.transform(image)
                label = self.labels[idx]
                return image, label
            else:
                print(f"Unreadable image skipped at runtime: {img_path}")
                # Skip to next image
                idx = (idx + 1) % len(self.image_paths)
                attempt += 1
        raise RuntimeError("Too many unreadable images in a row.")


# Prepare data paths and labels lists
train_image_paths, train_labels = get_image_paths_and_labels(TRAIN_PATH)
val_image_paths, val_labels = get_image_paths_and_labels(VAL_PATH)
test_image_paths, test_labels = get_image_paths_and_labels(TEST_PATH)
```

```
# Transforms remain the same
data_transforms = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

# Create datasets properly
train_dataset = ISICDataset(train_image_paths, train_labels,
transform=data_transforms)
val_dataset = ISICDataset(val_image_paths, val_labels, transform=data_transforms)
test_dataset = ISICDataset(test_image_paths, test_labels,
transform=data_transforms)

# Create dataloaders
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True,
num_workers=12, pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=128, shuffle=False,
num_workers=12, pin_memory=True)
```

```
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False,
num_workers=12, pin_memory=True)

print("DataLoaders created successfully.")



# Test the loader

sample_images, sample_labels = next(iter(train_loader))

print(f"Sample Images Shape: {sample_images.shape}")

print(f"Sample Labels Shape: {sample_labels.shape}")
```

APPENDIX C

THE PROPOSED MODEL (BERSERKER)

```
# LETS GO BERSERK!

class Berserker(nn.Module):

    def __init__(self, num_classes):

        super(Berserker, self).__init__()



        # ResNet50 Backbone

        resnet = models.resnet50(weights="IMAGENET1K_V1")

        self.resnet_feat = nn.Sequential(*list(resnet.children())[:-2])



        # EfficientNetB4 Backbone va Extract features from the penultimate layer

        efficientnet = create_model('efficientnet_b4', pretrained=True)

        self.efficientnet_feat = nn.Sequential(*list(efficientnet.children())[:-2])  # It removes classifier (keeps features)



        # DenseNet201 Backbone

        densenet = models.densenet201(weights="IMAGENET1K_V1")
```

```
        self.densenet_feat = densenet.features

        # Swin Transformer Backbone
        swin = create_model('swin_base_patch4_window7_224', pretrained=True,
features_only=True)
        self.swin_feat = swin

        # Freeze backbone layers to prevent them from training
        for param in self.resnet_feat.parameters():
            param.requires_grad = False

        for param in self.efficientnet_feat.parameters():
            param.requires_grad = False

        for param in self.densenet_feat.parameters():
            param.requires_grad = False

        for param in self.swin_feat.parameters():
            param.requires_grad = False
```

```
# Global Average Pooling
self.global_pool = nn.AdaptiveAvgPool2d((1, 1))

# Calculate the total input size dynamically for the fully connected layer
fc_input_size = 0

# Function to calculate the feature size from a model
def calculate_fc_input_size(model, dummy_input):
    with torch.no_grad():
        output = model(dummy_input)
        # If the model output is a list, get the first tensor
        if isinstance(output, list):
            output = output[0]
        output = self.global_pool(output)
        output = torch.flatten(output, 1)
        return output.shape[1]
```

```
# Create a dummy input to calculate the feature sizes
dummy_input = torch.randn(1, 3, 224, 224)  # Create a dummy input

# Calculate the input size for each backbone
fc_input_size += calculate_fc_input_size(self.resnet_feat, dummy_input)
fc_input_size += calculate_fc_input_size(self.efficientnet_feat, dummy_input)
fc_input_size += calculate_fc_input_size(self.densenet_feat, dummy_input)
fc_input_size += calculate_fc_input_size(self.swin_feat, dummy_input)

# Calculate the total input size for the fully connected layer
fc_input_size = 2048 + 1792 + 1920 + 7

# Classification Head
self.fc = nn.Sequential(
    nn.Linear(fc_input_size, 512),
    nn.ReLU(inplace=True),
    nn.Dropout(0.5),
    nn.Linear(512, num_classes)
```

```
    )


def forward(self, x):
    # Extract features from ResNet
    resnet_out = self.resnet_feat(x)
    resnet_out = self.global_pool(resnet_out)
    resnet_out = torch.flatten(resnet_out, 1)

    # Extract features from EfficientNet
    efficientnet_out = self.efficientnet_feat(x)
    efficientnet_out = self.global_pool(efficientnet_out)
    efficientnet_out = torch.flatten(efficientnet_out, 1)

    # Extract features from DenseNet
    densenet_out = self.densenet_feat(x)
    densenet_out = self.global_pool(densenet_out)
    densenet_out = torch.flatten(densenet_out, 1)
```

```
# Extract features from Swin Transformer
swin_feats = self.swin_feat(x)[-1]        # Get last feature map
swin_out = self.global_pool(swin_feats)   # (B, C, 1, 1)
swin_out = torch.flatten(swin_out, 1)     # Now (B, C)


# Concatenate all features
combined_features = torch.cat(
    (resnet_out, efficientnet_out, densenet_out, swin_out), dim=1
)


# Classification head
out = self.fc(combined_features)
return out
```

APPENDIX D

TRAINING

```
# This is the code cell for training the BERSERKER! (copied from the file TrainAndTestISIC2020.ipynb)

NUM_CLASSES = 2

# Only one checkpoint path for best model

best_checkpoint_dir = "/content/drive/MyDrive/THESIS/Dataset/kaggle/trainAndTest_from_train"

os.makedirs(best_checkpoint_dir, exist_ok=True)

best_checkpoint_path = os.path.join(best_checkpoint_dir, "only_2020.pth")



model = Berserker(num_classes=NUM_CLASSES)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model.to(device)



# Optimizer & Loss

optimizer = optim.AdamW(

    filter(lambda p: p.requires_grad, model.parameters()),

    lr=1e-4,

    weight_decay=1e-4

)

criterion = nn.CrossEntropyLoss()
```

```
start_epoch = 0


# No loading from previous checkpoint, start fresh every time.


def train(model, train_loader, val_loader, optimizer, criterion, num_epochs=10,
patience=3):
    best_val_loss = float('inf')
    epochs_no_improve = 0

    for epoch in range(start_epoch, num_epochs):
        print("Running epoch:", epoch + 1)  # Debug print
        model.train()
        running_loss, correct, total = 0.0, 0, 0

        for batch_idx, (inputs, labels) in enumerate(train_loader):
            print(f"Batch {batch_idx+1}/{len(train_loader)}")  # Debug print
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()

            outputs = model(inputs)
            loss = criterion(outputs, labels)
```

```
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, preds = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (preds == labels).sum().item()

    train_acc = 100 * correct / total
    train_loss = running_loss / len(train_loader)

    # Validation
    model.eval()
    val_loss, val_correct, val_total = 0.0, 0, 0

    with torch.no_grad():
        for inputs, labels in val_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
```

```
            val_loss += loss.item()
            _, preds = torch.max(outputs, 1)
            val_total += labels.size(0)
            val_correct += (preds == labels).sum().item()

    val_acc = 100 * val_correct / val_total
    val_loss_avg = val_loss / len(val_loader)

    print(f"[Epoch {epoch+1}/{num_epochs}] Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}% | Val Loss: {val_loss_avg:.4f} | Val Acc: {val_acc:.2f}%")

    if val_loss_avg < best_val_loss:
        best_val_loss = val_loss_avg
        epochs_no_improve = 0
        print(f"Validation loss improved. Saving best checkpoint to {best_checkpoint_path}")
        torch.save({
            'epoch': epoch + 1,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict()
        }, best_checkpoint_path)
    else:
        epochs_no_improve += 1
```

```
        if epochs_no_improve >= patience:
            print(f"Early stopping after {epoch+1} epochs due to no improvement.")
            break

# Launch training
train(model, train_loader, val_loader, optimizer, criterion, num_epochs=10,
patience=3)
```

APPENDIX E

TESTING (WITH TTA AND THRESHOLD FINDING)

```
# Testing with TTA (this cell is copied from the file train_20and19_test_19.ipynb)
# Finding threshold (first ran the testing one time before it)
precisions, recalls, thresholds = precision_recall_curve(all_labels, all_probs)


plt.plot(thresholds, precisions[:-1], label="Precision")
plt.plot(thresholds, recalls[:-1], label="Recall")
plt.xlabel("Threshold")
plt.ylabel("Score")
plt.title("Precision vs Recall vs Threshold")
plt.legend()
plt.grid(True)
plt.show()


f1_scores = 2 * (precisions * recalls) / (precisions + recalls + 1e-8)
best_idx = argmax(f1_scores)
print(f"Best F1 score: {f1_scores[best_idx]:.4f} at threshold
{thresholds[best_idx]:.2f}")


device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Berserker(num_classes=2)
model.to(device)


best_checkpoint_path =
"/content/drive/MyDrive/THESIS/Dataset/kaggle/best_19and20_train.pth"
checkpoint = torch.load(best_checkpoint_path, map_location=device)
```

```
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()

# transforms
tta_transforms = [
    T.Compose([]),  # original
    T.Compose([T.RandomHorizontalFlip(p=1.0)]),
    T.Compose([T.RandomVerticalFlip(p=1.0)]),
]

all_preds, all_labels, all_probs = [], [], []

with torch.no_grad():
    for images, labels in tqdm(test_loader, desc="Testing"):
        images, labels = images.to(device), labels.to(device)

        batch_size = images.size(0)
        probs_tta = torch.zeros(batch_size, 2).to(device)  # for accumulating TTA probs

        # Apply each TTA transform on the batch
        for transform in tta_transforms:
            # Transform expects PIL Image or Tensor in [C,H,W], here images are tensors.
            # Since images come as batches, apply transform per image
            augmented_images = torch.stack([transform(img.cpu()) for img in images]).to(device)

            outputs = model(augmented_images)
```

```
            probs = torch.softmax(outputs, dim=1)
            probs_tta += probs

        probs_tta /= len(tta_transforms)  # average probs over all TTA versions

        threshold = 0.43  # threshold for melanoma prob
        preds = (probs_tta[:, 1] > threshold).long()

        all_probs.extend(probs_tta[:, 1].cpu().numpy())
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

# Set averaging method based on output shape
average_type = 'binary' if model.fc[-1].out_features == 2 else 'weighted'

# Metrics
acc = accuracy_score(all_labels, all_preds)
precision = precision_score(all_labels, all_preds, average=average_type,
zero_division=0)
recall = recall_score(all_labels, all_preds, average=average_type, zero_division=0)
f1 = f1_score(all_labels, all_preds, average=average_type, zero_division=0)
cm = confusion_matrix(all_labels, all_preds)

print(f"\nTest Accuracy: {acc:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall (Sensitivity): {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
print("\nClassification Report:")
```

```
print(classification_report(all_labels, all_preds, target_names=["Benign",
"Melanoma"]))

print("\nConfusion Matrix:")

print(cm)


disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Benign",
"Melanoma"])

disp.plot(cmap=plt.cm.Blues)

plt.title("Confusion Matrix on Test Set (with TTA)")

plt.savefig("confusion_matrix_test_tta.png")

plt.show()


if model.fc[-1].out_features == 2:
    auc = roc_auc_score(all_labels, all_probs)
    print(f"AUC: {auc:.4f}")


    fpr, tpr, _ = roc_curve(all_labels, all_probs)
    plt.plot(fpr, tpr, label=f"AUC = {auc:.4f}")
    plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("ROC Curve (with TTA)")
    plt.legend()
    plt.grid(True)
    plt.savefig("roc_curve_test_tta.png")
    plt.show()
```

APPENDIX F

PREPROCESSING

```
# From the files TrainAndTestISIC2019_2ndTRY.ipynb,
train_20and19_test_19.ipynb and TrainAndTestISIC2020.ipynb

# Splitting New Folders

# Source directories

DATASET_PATH = "/content/drive/MyDrive/THESIS/Dataset/kaggle"

TRAIN_IMAGES_PATH = os.path.join(DATASET_PATH, "train")

TRAIN_BENIGN_PATH = os.path.join(TRAIN_IMAGES_PATH, "Benign")

TRAIN_MELANOMA_PATH = os.path.join(TRAIN_IMAGES_PATH,
"Melanoma")



# Destination base path

DEST_BASE_PATH = os.path.join(DATASET_PATH, "trainAndTest_from_train")

classes = ["Benign", "Melanoma"]



# Split ratios

train_ratio = 0.7

val_ratio = 0.15
```

```
test_ratio = 0.15


# Create output directories
for split in ['train', 'val', 'test']:
    for cls in classes:
        Path(os.path.join(DEST_BASE_PATH, split, cls)).mkdir(parents=True, exist_ok=True)


# Helper function to split and copy files
def split_and_copy(class_name, src_folder):
    images = os.listdir(src_folder)
    random.shuffle(images)

    total = len(images)
    train_end = int(train_ratio * total)
    val_end = train_end + int(val_ratio * total)

    split_data = {
        "train": images[:train_end],
```

```
        "val": images[train_end:val_end],
        "test": images[val_end:]
    }

    for split, file_list in split_data.items():
        for file in file_list:
            src = os.path.join(src_folder, file)
            dst = os.path.join(DEST_BASE_PATH, split, class_name, file)
            shutil.copyfile(src, dst)

# Perform the split
split_and_copy("Benign", TRAIN_BENIGN_PATH)
split_and_copy("Melanoma", TRAIN_MELANOMA_PATH)
print("Splitting complete. Check the 'trainAndTest_from_train' directory.")

# Counting for Checking
# Base path where your split dataset is stored
base_path =
"/content/drive/MyDrive/THESIS/Dataset/kaggle/trainAndTest_from_train"
```

```
splits = ["train", "val", "test"]

classes = ["Benign", "Melanoma"]



print("Image Counts:")

for split in splits:

    for cls in classes:

        folder = os.path.join(base_path, split, cls)

        count = len([f for f in os.listdir(folder) if f.lower().endswith(('.png', '.jpg', '.jpeg'))])

        print(f"{split.upper()} - {cls}: {count} images")



# Convert ISIC 2019 Training Data to Binary

# Paths

csv_path = '/content/drive/MyDrive/THESIS/Dataset/ISIC_2019_Train/ISIC_2019_Training_GroundTruth.csv'

img_dir = '/content/drive/MyDrive/THESIS/Dataset/ISIC_2019_Train/ISIC_2019_Training_Input'
```

```
output_dir =
'/content/drive/MyDrive/THESIS/Dataset/ISIC_2019_Train/ISIC_2019_Training_In
put'


# Create output folders

mel_dir = os.path.join(output_dir, 'Melanoma')

ben_dir = os.path.join(output_dir, 'Benign')

os.makedirs(mel_dir, exist_ok=True)

os.makedirs(ben_dir, exist_ok=True)


# Load ground truth

df = pd.read_csv(csv_path)


# Iterate and move

moved = {'Melanoma': 0, 'Benign': 0}

for idx, row in df.iterrows():

    image_id = row['image']

    is_melanoma = row['MEL'] == 1

    file_name = image_id + ".jpg"
```

```
    src_path = os.path.join(img_dir, file_name)

    if not os.path.exists(src_path):
        print(f"Missing: {src_path}")
        continue

    dst_path = os.path.join(mel_dir if is_melanoma else ben_dir, file_name)
    shutil.move(src_path, dst_path)
    moved['Melanoma' if is_melanoma else 'Benign'] += 1

print(f"Done. Moved {moved['Melanoma']} melanoma images and
{moved['Benign']} benign images.")

# Counting for Checking
# Base path where your split dataset is stored
mel_dir =
'/content/drive/MyDrive/THESIS/Dataset/ISIC_2019_Train/ISIC_2019_Training_In
put/Melanoma'
```

```
ben_dir =
'/content/drive/MyDrive/THESIS/Dataset/ISIC_2019_Train/ISIC_2019_Training_In
put/Benign'


# Count function
def count_images(folder):
    valid_exts = ('.jpg', '.jpeg', '.png')
    return len([f for f in os.listdir(folder) if f.lower().endswith(valid_exts)])


# Count and print
mel_count = count_images(mel_dir)
ben_count = count_images(ben_dir)


print(f"Melanoma: {mel_count}")
print(f"Benign: {ben_count}")


# Augmentation for ISIC 2019 Training Set
```

```
# CONFIG

melanoma_dir =
"/content/drive/MyDrive/THESIS/Dataset/ISIC_2019_Train/ISIC_2019_Training_In
put/Melanoma"

metadata_path =
'/content/drive/MyDrive/THESIS/Dataset/ISIC_2019_Train/ISIC_2019_Training_Gr
oundTruth.csv'

image_ext = ".jpg"


# LOAD METADATA

metadata_df = pd.read_csv(metadata_path, delimiter='\t' if '\t' in
open(metadata_path).readline() else ',')  # auto-detect delimiter


melanoma_df = metadata_df[metadata_df['MEL'] == 1]

original_count = len(melanoma_df)

target_count = 20809

needed_augments = target_count - original_count

aug_per_image = (needed_augments // original_count) + 1

print(f"Original Melanoma Images: {original_count}")

print(f"Target: {target_count} → Need to generate {needed_augments} new samples
(≈ {aug_per_image} per image)\n")
```

```
# AUGMENTATION TRANSFORMS
transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.5),
    A.RandomBrightnessContrast(p=0.5),
    A.Rotate(limit=20, p=0.5),
    A.ShiftScaleRotate(shift_limit=0.1, scale_limit=0.1, rotate_limit=10, p=0.5),
    A.Resize(224, 224),
])

# AUGMENT AND SAVE
new_rows = []
counter = 0

for _, row in tqdm(melanoma_df.iterrows(), total=original_count):
    image_id = row['image']
    original_path = os.path.join(melanoma_dir, f"{image_id}{image_ext}")
    image = cv2.imread(original_path)
```

```
if image is None:
    print(f"Could not read {original_path}")
    continue

for i in range(aug_per_image):
    if counter >= needed_augments:
        break

    augmented = transform(image=image)['image']
    new_id = f"{image_id}_aug{i}"
    save_path = os.path.join(melanoma_dir, f"{new_id}{image_ext}")
    cv2.imwrite(save_path, augmented)

    # Construct new metadata row
    new_row = {col: 0 for col in metadata_df.columns}
    new_row['image'] = new_id
    new_row['MEL'] = 1
    new_rows.append(new_row)
```

```
        counter += 1


    if counter >= needed_augments:
        break


# APPEND TO METADATA
augmented_df = pd.DataFrame(new_rows)
combined_df = pd.concat([metadata_df, augmented_df], ignore_index=True)


# SAVE FINAL METADATA
combined_df.to_csv(metadata_path, index=False)
print(f"\n{counter} augmented melanoma images added.")
print(f"Metadata updated at: {metadata_path}")


# Normalize ISIC 2019 Images
# Config
input_root =
'/content/drive/MyDrive/THESIS/Dataset/ISIC_2019_Train/ISIC_2019_Training_In
put'
```

```
output_root = '/content/drive/MyDrive/THESIS/Dataset/ISIC_2019_Train/ISIC_2019_Training_Input_Normalized'

image_size = (224, 224)

valid_exts = ('.jpg', '.png', '.jpeg')


mel_dir = os.path.join(input_root, 'Melanoma')

ben_dir = os.path.join(input_root, 'Benign')


# Create output directories

os.makedirs(os.path.join(output_root, 'Melanoma'), exist_ok=True)

os.makedirs(os.path.join(output_root, 'Benign'), exist_ok=True)


# Normalize function

def normalize_and_save(cls):

    input_dir = os.path.join(input_root, cls)

    output_dir_cls = os.path.join(output_root, cls)


    for fname in tqdm(os.listdir(input_dir), desc=f'Processing {cls}'):
```

```
if not fname.lower().endswith(valid_exts):
    continue
input_path = os.path.join(input_dir, fname)
output_path = os.path.join(output_dir_cls, fname)

image = cv2.imread(input_path)
if image is None:
    print(f"Could not read: {input_path}")
    continue

# Resize
image = cv2.resize(image, image_size)

# BGR → RGB
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Normalize to [0, 1]
image = image.astype(np.float32) / 255.0
```

```
        # Save as uint8 (for storage); training will divide by 255 again
        save_image = (image * 255).astype(np.uint8)
        save_image = cv2.cvtColor(save_image, cv2.COLOR_RGB2BGR)
        cv2.imwrite(output_path, save_image)

# Run for both classes
normalize_and_save('Melanoma')
normalize_and_save('Benign')
print(f"\nNormalized images saved to: {output_root}")
```